

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Avantio Fierza Patria NIM. 2310817310001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN Mobile

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Avantio Fierza Patria
NIM : 2310817310001

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom.,
M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN.....	1
DAFTAR ISI	2
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
MODUL 1: Android Basics with Compose	7
A. Source Code menggunakan Jetpack Compose	9
B. Source Code menggunakan XML	12
C. Output Program	17
D. Pembahasan	17
MODUL 2: ANDROID LAYOUT WITH COMPOSE	21
Soal Praktikum:	21
A. Source Code menggunakan Jetpack Compose	23
B. Source Code menggunakan XML	25
C. Output Program	28
D. Pembahasan	30
MODUL 3: BUILD A SCROLLABLE LIST	34
Soal Praktikum:	34
A. Source Code menggunakan Jetpack Compose	35
B. Source Code menggunakan XML	36
C. Output Program	37
D. Pembahasan	38
MODUL 4: VIEWMODEL AND DEBUGGING	40
Soal Praktikum:	40
A. Source Code menggunakan Jetpack Compose	40
B. Source Code menggunakan XML	49
C. Output Program	59
D. Pembahasan	62
MODUL 5: CONNECT TO THE INTERNET	65
Soal Praktikum:	65
A. Source Code menggunakan Jetpack Compose	65
B. Output Program	82
C. Pembahasan	83

E. Tautan GIT	87
---------------------	----

DAFTAR GAMBAR

MODUL 1: ANDROID BASICS WITH COMPOSE

Gambar 1.1 1 Tampilan Awal Aplikasi.....	7
Gambar 1.1 2 Tampilan Dadu Setelah Di Roll.....	8
Gambar 1.1 3 Tampilan Roll Dadu Double.....	8
Gambar 1.1 4 Screenshot Hasil Jawaban Soal 1	17

MODUL 2: ANDROID LAYOUT WITH COMPOSE

Gambar 2.1 1 Gambar Tampilan awal aplikasi	21
Gambar 2.1 2 Tampilan Pilihan Persentase Tip	22
Gambar 2.1 3 Tampilan Aplikasi Setelah Dijalankan	22
Gambar 2.1 4 Screenshot Tampilan Awal.....	28
Gambar 2.1 5 Screenshot Tampilan Pilihan Tip.....	29
Gambar 2.1 6 Screenshot Aplikasi Setelah Dijalankan.....	30

MODUL 3: BUILD A SCROLLABLE LIST

Gambar 3.1 1 Contoh UI List	34
Gambar 3.1 2 Contoh UI Detail.....	35
Gambar 3.1 3 Screenshot Tampilan UI List	37
Gambar 3.1 4 Screenshot Tampilan UI Detail	38

MODUL 4: VIEWMODEL AND DEBUGGING

Gambar 4.1 1 Contoh Penggunaan Debugger	40
Gambar 4.1 2 Screenshot Debugging Step Into Jetpack Compose	59
Gambar 4.1 3 Screenshot Debugging Step Over Jetpack Compose.....	60
Gambar 4.1 4 Screenshot Debugging Step Out Jetpack Compose.....	60
Gambar 4.1 5 Screenshot Debugging Step Into XML	61
Gambar 4.1 6 Screenshot Debugging Step Over XML	61
Gambar 4.1 7 Screenshot Debugging Step Out XML	62

MODUL 5: CONNECT TO THE INTERNET

Gambar 5.1 1 Screenshot Hasil Jawaban Soal 1 Modul 5.....	82
---	----

DAFTAR TABEL

MODUL 1: ANDROID BASICS WITH COMPOSE

Tabel 1.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 1	12
Tabel 1.1 2 Source Code Jawaban Soal 1 MainActivity.kt XML Modul 1	14
Tabel 1.1 3 Source Code Jawaban Soal 1 actibity_maint .XML Modul 1	16

MODUL 2: ANDROID LAYOUT WITH COMPOSE

Tabel 2.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 2	25
Tabel 2.1 2 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 2	26
Tabel 2.1 3 Source Code Jawaban Soal 1 actibity_maint .XML Modul 2	27

MODUL 3: BUILD A SCROLLABLE LIST

Tabel 3.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 3	35
Tabel 3.1 2 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 3	36
Tabel 3.1 3 Source Code Jawaban Soal 1 activity_maint .XML Modul 3	36

MODUL 4: VIEWMODEL AND DEBUGGING

Tabel 4.1 1 Source Code MainActivity.kt Jetpack Compose Modul 4	41
Tabel 4.1 2 Source Code Book.kt Jetpack Compose	41
Tabel 4.1 3 Source Code BookViewModel.kt Jetpack Compose	44
Tabel 4.1 4 Source Code DetailScreen.kt Jetpack Compose	45
Tabel 4.1 5 Source Code HomeScreen.kt Jetpack Compose	48
Tabel 4.1 6 Source Code MyApplication.kt Jetpack Compose	48
Tabel 4.1 7 Source Code Navigation.kt Jetpack Compose	49
Tabel 4.1 8 Source Code MainActivity.kt XML Modul 4	49
Tabel 4.1 9 Source Code Book.kt XML	50
Tabel 4.1 10 Source Code BookAdapter.kt XML	51
Tabel 4.1 11 Source Code BookDetailFragment.kt XML	51
Tabel 4.1 12 Source Code BookListFragment.kt XML	52
Tabel 4.1 13 Source Code BookViewModel.kt XML	55
Tabel 4.1 14 Source Code BookViewModelFactory.kt XML	55
Tabel 4.1 15 Source Code activity_maint .XML	56
Tabel 4.1 16 Source Code fragment_book_detail .XML	57
Tabel 4.1 17 Source Code fragment_book_list .XML	57
Tabel 4.1 18 Source Code item_book_ .XML	59

MODUL 5: CONNECT TO THE INTERNET

Tabel 5.1 1 Source Code MainActivity.kt Jetpack Compose Modul 5	65
Tabel 5.1 2 Source Code ApiService.kt Jetpack Compose	66
Tabel 5.1 3 Source Code DetailScreen.kt Jetpack Compose	68
Tabel 5.1 4 Source Code Film.kt Jetpack Compose	68
Tabel 5.1 5 Source Code FilmRepository.kt Jetpack Compose	68
Tabel 5.1 6 Source Code FilmRepository.kt Jetpack Compose	69

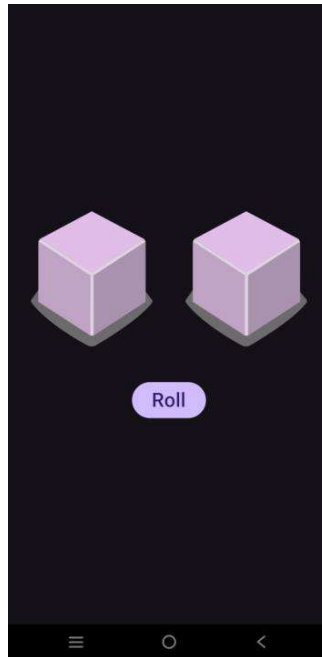
Tabel 5.1 7 Source Code FlmViewModelFactory.kt Jetpack Compose.....	70
Tabel 5.1 8 Source Code HomeScreen.kt Jetpack Compose.....	73
Tabel 5.1 9 Source Code MainScreen.kt Jetpack Compose	74
Tabel 5.1 10 Source Code MovieResponse.kt Jetpack Compose	74
Tabel 5.1 11 Source Code MovieResponse.kt Jetpack Compose	75
Tabel 5.1 12 Source Code MyApplication.kt Jetpack Compose.....	76
Tabel 5.1 13 Source Code Navigation.kt Jetpack Compose.....	77
Tabel 5.1 14 Source Code NetworkFilm.kt Jetpack Compose.....	77
Tabel 5.1 15 Source Code PreferenceManager.kt Jetpack Compose.....	78
Tabel 5.1 16 Source Code Result.kt Jetpack Compose	78
Tabel 5.1 17 Source Code RetrofitInstance.kt Jetpack Compose	79
Tabel 5.1 18 Source Code SharedPreferenceViewModel.kt Jetpack Compose.....	79
Tabel 5.1 19 Source Code SharedPreferenceViewModelFactory.kt Jetpack Compose	80
Tabel 5.1 20 Source Code TMDBApi.kt Jetpack Compose.....	80
Tabel 5.1 21 Source Code SettingScreen.kt Jetpack Compose	81

MODUL 1: Android Basics with Compose

Soal Praktikum:

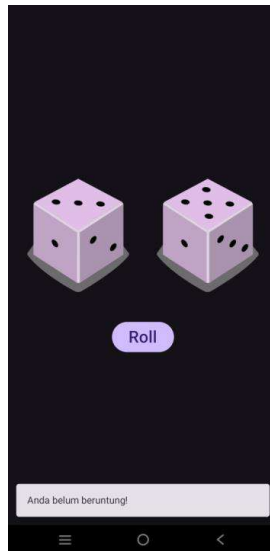
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



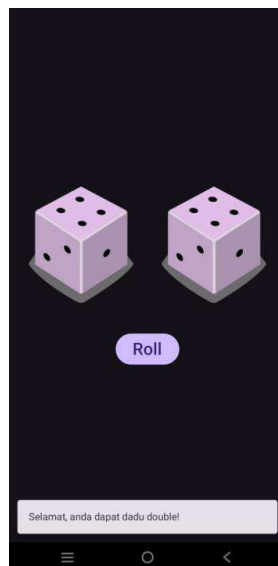
Gambar 1.1 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 1.1 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 1.1 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.

5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam folder Modul 1 dalam bentuk Project. Jangan lupa untuk melakukan Clean Project sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:

https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code menggunakan Jetpack Compose MainActivity.kt

```
1 package com.example.diceroller
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.Image
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.res.painterResource
14 import androidx.compose.ui.unit.dp
15 import androidx.compose.ui.unit.sp
16 import kotlin.random.Random
17
18 class MainActivity : ComponentActivity() {
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         setContent {
22             DiceRollerApp()
23         }
24     }
```

```

25 }
26
27 @Composable
28 fun DiceRollerApp() {
29     MaterialTheme {
30         Surface(
31             modifier = Modifier.fillMaxSize(),
32             color = Color(0xFFFFF5E1) // Warna krem
33         ) {
34             DiceContent()
35         }
36     }
37 }
38
39 @Composable
40 fun DiceContent() {
41     var dice1Value by remember { mutableStateOf(1) }
42     var dice2Value by remember { mutableStateOf(1) }
43     var message by remember { mutableStateOf("Tekan tombol
44 untuk mulai!") }
45
46     Column(
47         modifier = Modifier
48             .fillMaxSize()
49             .padding(16.dp),
50         horizontalAlignment = Alignment.CenterHorizontally,
51         verticalArrangement = Arrangement.Center
52     ) {
53         Text(text = "Roll the Dice!", fontSize = 24.sp, color
54 = Color.Black)
55
56         Spacer(modifier = Modifier.height(16.dp))

```

```

56
57         Row(
58             horizontalArrangement = Arrangement.Center
59         ) {
60             DiceImage(dice1Value)
61             Spacer(modifier = Modifier.width(16.dp))
62             DiceImage(dice2Value)
63         }
64
65         Spacer(modifier = Modifier.height(16.dp))
66
67         Text(text = message, fontSize = 18.sp, color =
68         Color.DarkGray)
69
70         Spacer(modifier = Modifier.height(24.dp))
71
72         Button(
73             onClick = {
74                 dice1Value = Random.nextInt(1, 7)
75                 dice2Value = Random.nextInt(1, 7)
76
77                 message = if (dice1Value == dice2Value) {
78                     "🎉 Selamat, anda dapat dadu double! 🎉"
79                 } else {
80                     "Anda belum beruntung! 😞"
81                 }
82             }
83         ) {
84             Text(text = "Roll", fontSize = 18.sp)
85         }
86     }

```

87	}
88	
89	@Composable
90	fun DiceImage(diceValue: Int) {
91	Image(
92	painter = painterResource(id =
93	getDiceImageResource(diceValue)),
94	contentDescription = "Dice \$diceValue",
95	modifier = Modifier.size(100.dp)
96	)
97	}
98	
99	fun getDiceImageResource(value: Int): Int {
100	return when (value) {
101	1 -> R.drawable.dice_1
102	2 -> R.drawable.dice_2
103	3 -> R.drawable.dice_3
104	4 -> R.drawable.dice_4
105	5 -> R.drawable.dice_5
106	6 -> R.drawable.dice_6
107	else -> R.drawable.dice_1
108	}
109	}

Tabel 1.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 1

B. Source Code menggunakan XML

MainActivity.kt

1	package com.example.diceroller2
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView

```

6 import android.widget.TextView
7 import androidx.appcompat.app.AppCompatActivity
8 import kotlin.random.Random
9
10 class MainActivity : AppCompatActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContentView(R.layout.activity_main)
14
15         // Ambil referensi dari UI
16         val dice1Image: ImageView =
17         findViewById(R.id.dice1Image)
18         val dice2Image: ImageView =
19         findViewById(R.id.dice2Image)
20         val btnRoll: Button = findViewById(R.id.btnRoll)
21         val tvMessage: TextView = findViewById(R.id.tvMessage)
22
23         btnRoll.setOnClickListener {
24             val dice1Value = Random.nextInt(1, 7)
25             val dice2Value = Random.nextInt(1, 7)
26
27             // Update gambar dadu
28             dice1Image.setImageResource(getDiceImage(dice1Value))
29             dice2Image.setImageResource(getDiceImage(dice2Value))
30
31             // Update pesan berdasarkan hasil
32             tvMessage.text = if (dice1Value == dice2Value) {
33                 "🎉 Selamat, anda dapat dadu double! 🎉"
34             } else {
35                 "😞 Anda belum beruntung!"
36             }
37         }
38     }
39 }

```

37	}
38	}
39	
40	private fun getDiceImage(value: Int): Int {
41	return when (value) {
42	1 -> R.drawable.dice_1
43	2 -> R.drawable.dice_2
44	3 -> R.drawable.dice_3
45	4 -> R.drawable.dice_4
46	5 -> R.drawable.dice_5
47	6 -> R.drawable.dice_6
48	else -> R.drawable.dice_1
49	}
50	}
51	}

Tabel 1.1 2 Source Code Jawaban Soal 1 MainActivity.kt XML Modul 1

activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:background="@color/cream"
8	android:gravity="center"
9	android:padding="16dp">
10	
11	<!-- Judul -->
12	<TextView
13	android:id="@+id/tvTitle"

```

14         android:layout_width="wrap_content"
15         android:layout_height="wrap_content"
16         android:text="Roll the Dice!"
17         android:textSize="24sp"
18         android:textStyle="bold"
19         android:textColor="@color/black"
20         android:layout_marginBottom="16dp"/>
21
22     <!-- Dadu -->
23     <LinearLayout
24         android:layout_width="wrap_content"
25         android:layout_height="wrap_content"
26         android:orientation="horizontal"
27         android:gravity="center">
28
29         <ImageView
30             android:id="@+id/dice1Image"
31             android:layout_width="100dp"
32             android:layout_height="100dp"
33             android:src="@drawable/dice_1"/>
34
35         <Space
36             android:layout_width="16dp"
37             android:layout_height="wrap_content"/>
38
39         <ImageView
40             android:id="@+id/dice2Image"
41             android:layout_width="100dp"
42             android:layout_height="100dp"
43             android:src="@drawable/dice_1"/>
44     </LinearLayout>

```


45	
46	<!-- Pesan -->
47	<TextView
48	android:id="@+id/tvMessage"
49	android:layout_width="wrap_content"
50	android:layout_height="wrap_content"
51	android:text="Tekan tombol untuk mulai!"
52	android:textSize="18sp"
53	android:textColor="@color/black"
54	android:layout_marginTop="16dp"/>
55	
56	<!-- Tombol -->
57	<Button
58	android:id="@+id/btnRoll"
59	android:layout_width="wrap_content"
60	android:layout_height="wrap_content"
61	android:text="Roll"
62	android:textSize="18sp"
63	android:layout_marginTop="24dp"/>
	</LinearLayout>

Tabel 1.1 3 Source Code Jawaban Soal 1 activity_maint .XML Modul 1

C. Output Program



Gambar 1.1 4 Screenshot Hasil Jawaban Soal 1

D. Pembahasan

MainActivity.kt Jetpack Compose:

1. Import Library

- **ComponentActivity** → Kelas dasar untuk aktivitas yang mendukung **Jetpack Compose**.
- **setContent {}** → Fungsi untuk mengganti tampilan menggunakan **Compose**.

- **Image** → Digunakan untuk menampilkan gambar dadu.
- **Surface** → Digunakan untuk membungkus elemen UI dengan warna latar belakang.
- **MaterialTheme** → Menerapkan tema Material3.
- **Random** → Menghasilkan angka acak (untuk dadu).

2. MainActivity

- **MainActivity** → Kelas utama yang menjalankan aplikasi.
- **onCreate** → Metode yang dipanggil saat aplikasi pertama kali dijalankan.
- **setContent { DiceRollerApp() }** → Menetapkan tampilan utama aplikasi dengan Composable DiceRollerApp().

3. Fungsi DiceRollerApp()

- **@Composable** → Menandakan fungsi ini sebagai bagian dari UI Jetpack Compose.
- **MaterialTheme** → Menerapkan tema Material3 ke seluruh aplikasi.
- **Surface** → Membuat latar belakang dengan warna krem (#FFF5E1).
- **DiceContent()** → Memanggil fungsi yang menampilkan konten utama aplikasi.

4. Fungsi DiceContent()

- **var dice1Value by remember { mutableStateOf(1) }**
 - Menyimpan nilai dadu pertama, menggunakan remember agar tetap tersimpan saat UI diperbarui.
- **var dice2Value by remember { mutableStateOf(1) }**
 - Menyimpan nilai dadu kedua.
- **var message by remember { mutableStateOf("Tekan tombol untuk mulai!") }**
 - Menyimpan pesan hasil roll dadu.

5. Struktur Layout dengan Column

- **Column { }** → Menyusun elemen UI secara vertikal.
- **modifier = Modifier.fillMaxSize().padding(16.dp)**
 - Mengisi layar penuh dan memberikan padding **16dp**
- **horizontalAlignment = Alignment.CenterHorizontally**
 - Menyusun elemen ke **tengah horizontal**.
- **verticalArrangement = Arrangement.Center**
 - Menyusun elemen ke **tengah vertikal**.

6. Menampilkan Teks dan Gambar Dadu

- **Text("Roll the Dice!")** → Menampilkan teks judul.
- **Spacer** → Memberikan jarak antar elemen.
- **Row { }** → Menyusun elemen UI secara **horizontal** untuk kedua dadu.
- **DiceImage(dice1Value)** → Memanggil fungsi untuk menampilkan gambar dadu pertama.

- **DiceImage(dice2Value)** → Memanggil fungsi untuk menampilkan gambar dadu kedua.
- 7. Menampilkan Pesan Hasil Roll**
- **Menampilkan pesan hasil roll dadu (message).**
 - **Warna abu-abu tua (Color.DarkGray)** agar lebih terlihat di latar krem.
- 8. Tombol “Roll” untuk Mengacak Dadu**
- **Button(onClick = { ... })** → Menjalankan aksi saat tombol ditekan.
 - **Random.nextInt(1, 7)** → Menghasilkan angka acak **1-6** untuk dadu.
 - **Cek apakah dadu double:**
 - Jika **sama** → tampilkan 🎉 Selamat, anda dapat dadu double! 🎉
 - Jika **berbeda** → tampilkan "Anda belum beruntung! 😞"
- 9. Fungsi DiceImage()**
- **Menampilkan gambar dadu berdasarkan nilai diceValue.**
 - **Ukuran gambar dadu = 100.dp.**
- 10. Fungsi getDiceImageResource**
- **Mengembalikan gambar dadu yang sesuai dengan angka yang muncul.**

MainActivity.kt XML:

- 1. Import Library**
 - **AppCompatActivity** → Kelas dasar untuk aktivitas berbasis XML dengan fitur kompatibilitas lama.
 - **ImageView** → Untuk menampilkan gambar dadu.
 - **Button** → Untuk tombol "Roll".
 - **TextView** → Untuk menampilkan pesan hasil roll dadu.
 - **Random** → Untuk menghasilkan angka acak (1-6).
- 2. MainActivity**
 - **MainActivity** → Kelas utama yang menjalankan aplikasi.
 - **onCreate** → Metode yang dipanggil saat aplikasi pertama kali dijalankan.
 - **setContentView(R.layout.activity_main)** → Menetapkan tampilan utama dari file XML **activity_main.xml**.
- 3. Menghubungkan Elemen UI dari XML**
 - **Menghubungkan elemen-elemen UI** dengan **findViewById()** agar bisa digunakan dalam kode.
 - **dice1Image** → Gambar dadu pertama.
 - **dice2Image** → Gambar dadu kedua.
 - **btnRoll** → Tombol untuk meroll dadu.
 - **tvMessage** → Teks yang menampilkan pesan setelah roll.

4. Menetapkan Aksi pada Tombol "Roll"

- **btnRoll.setOnClickListener {}** → Menjalankan aksi saat tombol Roll ditekan.
- **Random.nextInt(1, 7)** → Menghasilkan angka acak antara 1-6 untuk setiap dadu.
setImageResource(getDiceImage(dice1Value)) → Mengubah gambar dadu sesuai hasil.
- Cek apakah dadu double:
- **Jika nilai dadu sama** → Tampilkan "🎉 Selamat, anda dapat dadu double! 🎉"
- **Jika nilai berbeda** → Tampilkan "😞 Anda belum beruntung!"

5. Fungsi getDiceImage ()

- Mengembalikan gambar dadu yang sesuai dengan angka yang muncul.
- Misalnya, jika angka 3 keluar, akan mengembalikan R.drawable.dice_3 (gambar dadu 3).

6. activity_main.xml:

- **LinearLayout** → Tata letak vertikal untuk menyusun elemen dari atas ke bawah.
- **android:background="#FFF5E1"** → Mengatur warna latar belakang menjadi krem.
- **TextView (tvMessage)** → Menampilkan pesan hasil roll dadu.
- **ImageView (dice1Image & dice2Image)** → Menampilkan gambar dadu.
- **Button (btnRoll)** → Tombol untuk roll dadu.

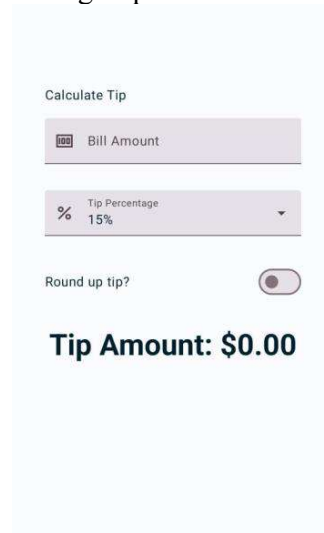
MODUL 2: ANDROID LAYOUT WITH COMPOSE

Soal Praktikum:

1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

- A. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- B. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- C. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- D. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.

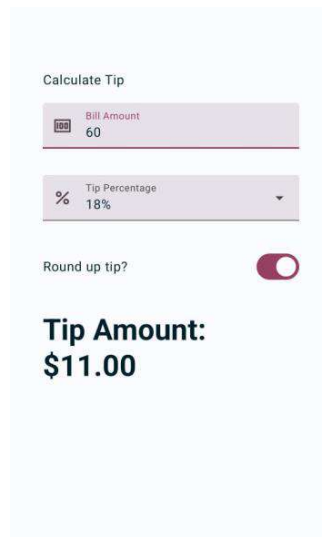
2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.



Gambar 2.1 1 Gambar Tampilan awal aplikasi



Gambar 2.1 2 Tampilan Pilihan Persentase Tip



Gambar 2.1 3 Tampilan Aplikasi Setelah Dijalankan

A. Source Code menggunakan Jetpack Compose

MainActivity.kt

```
1 package com.example.kalkulator
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.text.KeyboardOptions
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.text.input.KeyboardType
13 import androidx.compose.ui.unit.dp
14 import com.example.kalkulator.ui.theme.KalkulatorTheme
15 import kotlin.math.ceil
16
17 class MainActivity : ComponentActivity() {
18     override fun onCreate(savedInstanceState: Bundle?) {
19         super.onCreate(savedInstanceState)
20         setContent {
21             KalkulatorTheme {
22                 Surface(
23                     modifier = Modifier.fillMaxSize(),
24                     color
25                     MaterialTheme.colorScheme.background
26                 ) {
27                     TipCalculatorCompose()
28                 }
29             }
30         }
31     }
32 }
33
34 @OptIn(ExperimentalMaterial3Api::class)
35 @Composable
36 fun TipCalculatorCompose() {
37     var cost by remember { mutableStateOf("") }
38     var tipPercent by remember { mutableStateOf(15) }
39     var roundUp by remember { mutableStateOf(false) }
40     val tip = calculateTip(cost.toDoubleOrNull() ?: 0.0,
41 tipPercent, roundUp)
42
43     Column(
44         modifier = Modifier
45             .padding(16.dp)
46             .fillMaxWidth()
47     ) {
48         Text("Kalkulator Tip", style
49 MaterialTheme.typography.headlineSmall)
50
51         Spacer(modifier = Modifier.height(16.dp))
```



```

52
53         OutlinedTextField(
54             value = cost,
55             onValueChange = { cost = it },
56             label = { Text("Biaya Layanan") },
57             keyboardOptions = KeyboardOptions(keyboardType =
58 KeyboardType.Number),
59             modifier = Modifier.fillMaxWidth()
60         )
61
62         Spacer(modifier = Modifier.height(16.dp))
63
64         var expanded by remember { mutableStateOf(false) }
65         val tipOptions = listOf(10, 15, 20, 25)
66
67         ExposedDropdownMenuBox(
68             expanded = expanded,
69             onExpandedChange = { expanded = !expanded }
70         ) {
71             OutlinedTextField(
72                 value = "$tipPercent%",
73                 onValueChange = {},
74                 readOnly = true,
75                 label = { Text("Persentase Tip") },
76                 trailingIcon = {
77 ExposedDropdownMenuDefaults.TrailingIcon(expanded) },
78                 modifier =
79 Modifier.menuAnchor().fillMaxWidth()
80             )
81
82             ExposedDropdownMenu(
83                 expanded = expanded,
84                 onDismissRequest = { expanded = false }
85             ) {
86                 tipOptions.forEach { percent ->
87                     DropdownMenuItem(
88                         text = { Text("$percent%") },
89                         onClick = {
90                             tipPercent = percent
91                             expanded = false
92                         }
93                     )
94                 }
95             }
96         }
97
98         Spacer(modifier = Modifier.height(16.dp))
99
100        Row(
101            verticalAlignment = Alignment.CenterVertically,
102            modifier = Modifier.fillMaxWidth()
103        ) {
104            Text("Round up tip?", modifier =
105 Modifier.weight(1f))

```

106	Switch(
107	checked = roundUp,
108	onCheckedChangeListener = { roundUp = it },
109	colors = SwitchDefaults.colors(
110	checkedThumbColor
111	MaterialTheme.colorScheme.primary
112	)
113	)
114	}
115	Spacer(modifier = Modifier.height(24.dp))
116	
117	Text(
118	text = "Jumlah Tip: Rp%.2f".format(tip),
119	style = MaterialTheme.typography.titleMedium
120	)
121	}
122	}
123	
124	fun calculateTip(amount: Double, tipPercent: Int, roundUp:
125	Boolean): Double {
126	var tip = amount * tipPercent / 100
127	if (roundUp) tip = ceil(tip)
128	return tip
	}

Tabel 2.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 2

B. Source Code menggunakan XML

MainActivity.kt

1	package com.example.kalkulator2
2	
3	import android.os.Bundle
4	import android.text.Editable
5	import android.text.TextWatcher
6	import android.view.View
7	import android.widget.*
8	import androidx.appcompat.app.AppCompatActivity
9	import kotlin.math.ceil
10	
11	class MainActivity : AppCompatActivity() {
12	private lateinit var costInput: EditText
13	private lateinit var tipSpinner: Spinner
14	private lateinit var roundUpSwitch: Switch
15	private lateinit var tipResult: TextView
16	
17	override fun onCreate(savedInstanceState: Bundle?) {
18	super.onCreate(savedInstanceState)
19	setContentView(R.layout.activity_main)
20	
21	costInput = findViewById(R.id.costInput)
22	tipSpinner = findViewById(R.id.tipSpinner)
23	roundUpSwitch = findViewById(R.id.roundUpSwitch)

24	tipResult = findViewById(R.id.tipResult)
25	
26	val tipOptions = listOf(10, 15, 20, 25)
27	val adapter = ArrayAdapter(this,
28	android.R.layout.simple_spinner_dropdown_item, tipOptions)
29	tipSpinner.adapter = adapter
30	
31	val updateTip = {
32	val cost =
33	costInput.text.toString().toDoubleOrNull() ?: 0.0
34	val tipPercent = tipSpinner.selectedItem as Int
35	val roundUp = roundUpSwitch.isChecked
36	
37	var tip = cost * tipPercent / 100
38	if (roundUp) tip = ceil(tip)
39	
40	tipResult.text = "Jumlah Tip: Rp%.2f".format(tip)
41	}
42	
43	costInput.addTextChangedListener(object : TextWatcher {
44	override fun afterTextChanged(s: Editable?) =
45	updateTip()
46	override fun beforeTextChanged(s: CharSequence?,
47	start: Int, count: Int, after: Int) {}
48	override fun onTextChanged(s: CharSequence?, start:
49	Int, before: Int, count: Int) {}
50	})
51	
52	roundUpSwitch.setOnCheckedChangeListener { _, _ ->
53	updateTip() }
54	
55	tipSpinner.onItemSelectedListener = object :
56	AdapterView.OnItemSelectedListener {
57	override fun onItemSelected(
58	parent: AdapterView<*>?, view: View?, position:
59	Int, id: Long
60	) = updateTip()
61	
62	override fun onNothingSelected(parent:
63	AdapterView<*>?) {}
64	}
65	}
66	}

Tabel 2.1 2 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 2

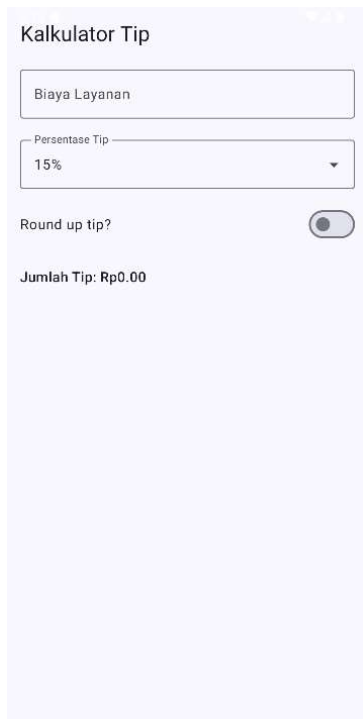
activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:orientation="vertical"
5	android:padding="16dp"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	

9	<TextView
10	android:text="Kalkulator Tip"
11	android:textSize="24sp"
12	android:textStyle="bold"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content" />
15	
16	<EditText
17	android:id="@+id/costInput"
18	android:hint="Biaya Layanan"
19	android:inputType="numberDecimal"
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content"
22	android:layout_marginTop="16dp"/>
23	
24	<Spinner
25	android:id="@+id/tipSpinner"
26	android:layout_width="match_parent"
27	android:layout_height="wrap_content"
28	android:layout_marginTop="16dp" />
29	
30	<LinearLayout
31	android:orientation="horizontal"
32	android:layout_width="match_parent"
33	android:layout_height="wrap_content"
34	android:layout_marginTop="16dp"
35	android:gravity="center_vertical">
36	
37	<TextView
38	android:text="Round up tip?"
39	android:layout_width="0dp"
40	android:layout_weight="1"
41	android:layout_height="wrap_content"/>
42	
43	<Switch
44	android:id="@+id/roundUpSwitch"
45	android:layout_width="wrap_content"
46	android:layout_height="wrap_content"
47	android:thumbTint="@android:color/white"/>
48	</LinearLayout>
49	
50	<TextView
51	android:id="@+id/tipResult"
52	android:text="Jumlah Tip: Rp0.00"
53	android:textSize="18sp"
54	android:textStyle="bold"
55	android:layout_marginTop="24dp"
56	android:layout_width="wrap_content"
57	android:layout_height="wrap_content"/>
	</LinearLayout>

Tabel 2.1 3 Source Code Jawaban Soal 1 activity_maint.XML Modul 2

C. Output Program



The screenshot shows a mobile application titled "Kalkulator Tip". It features a text input field labeled "Biaya Layanan". Below it is a dropdown menu labeled "Persentase Tip" with "15%" selected. A toggle switch labeled "Round up tip?" is currently turned off. At the bottom, it displays "Jumlah Tip: Rp0.00".

Gambar 2.1 4 Screenshot Tampilan Awal

Kalkulator Tip

Biaya Layanan

Persentase Tip

15%

10%

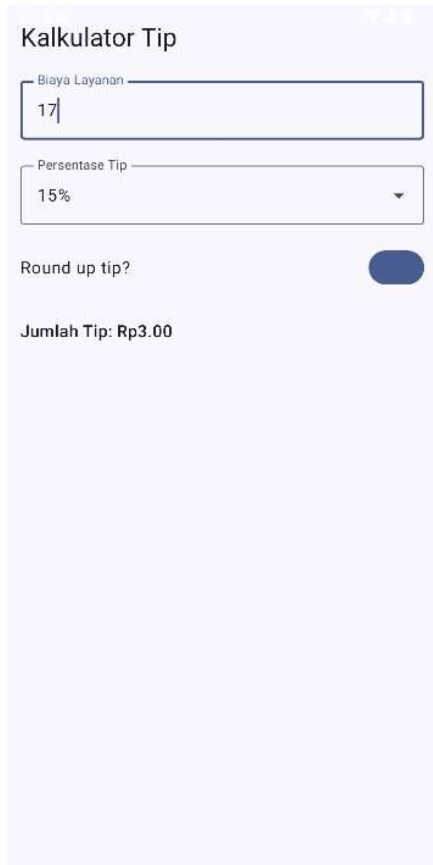
15%

20%

25%

☐

Gambar 2.1 5 Screenshot Tampilan Pilihan Tip



Gambar 2.1 6 Screenshot Aplikasi Setelah Dijalankan

D. Pembahasan

MainActivity.kt Jetpack Compose:

1. MainActivity

Di *MainActivity*, kita menggunakan komponen *ComponentActivity* karena Compose tidak lagi menggunakan layout XML. Di dalam fungsi *onCreate*, kita langsung menetapkan konten UI menggunakan fungsi *setContent*.

Seluruh tampilan aplikasi dibungkus dalam *Surface*, dan di dalamnya kita memanggil fungsi *TipCalculatorCompose()*, yaitu UI utama kita.

2. Fungsi *TipCalculatorCompose()*

Fungsi ini berisi seluruh antarmuka pengguna. Kita mendefinisikan beberapa state:

- *cost* menyimpan input biaya layanan dari pengguna.
- *tipPercent* menyimpan persentase tip yang dipilih.
- *roundUp* menyimpan status switch, apakah pengguna ingin membulatkan tip ke atas.

State ini memungkinkan UI untuk secara otomatis merespons perubahan nilai yang dilakukan pengguna.

3. Input Biaya Layanan

Bagian ini menggunakan OutlinedTextField, yaitu versi Compose dari input teks yang familiar di Material Design. Ketika pengguna mengetik, nilai cost akan berubah, dan tip akan dihitung ulang.

4. Dropdown Pilihan Tip (*Spinner*)

Kita menggunakan ExposedDropDownMenuBox, yang fungsinya mirip seperti Spinner di XML. Pengguna bisa mengetuknya untuk memilih persentase tip (10%, 15%, 20%, atau 25%). Saat dipilih, nilai tipPercent diperbarui.

5. Switch "Round Up Tip"

Untuk opsi membulatkan ke atas, kita menggunakan Switch() yang diletakkan dalam Row, berdampingan dengan teks deskriptif. Jika pengguna mengaktifkan switch ini, nilai tip akan dibulatkan ke atas menggunakan ceil().

6. Tampilan Hasil Tip

Di bagian bawah, kita menampilkan hasil perhitungan tip dalam format "Rp...", dan tampilannya akan berubah secara otomatis ketika input, persentase, atau switch diubah.

7. Fungsi calculateTip()

Fungsi ini menerima tiga parameter: jumlah biaya, persentase tip, dan status pembulatan. Fungsi ini melakukan perhitungan tip, dan jika roundUp bernilai true, maka nilai tip akan dibulatkan ke atas.

MainActivity.kt XML:

1. activity_main.xml

File ini berisi antarmuka pengguna menggunakan pendekatan tradisional. Komponen-komponen utama antara lain:

- EditText: Tempat pengguna memasukkan biaya layanan.
- Spinner: Digunakan untuk memilih persentase tip.
- Switch: Digunakan untuk memilih apakah ingin membulatkan ke atas.
- TextView: Menampilkan hasil tip.

View: Sebagai pemisah visual (divider) di bawah spinner agar tampilan lebih rapi.

2. MainActivity.kt

Di sini kita menggunakan pendekatan View-based (lama). Di dalam onCreate, kita menghubungkan semua komponen dari XML ke Kotlin menggunakan findViewById.

3. Inisialisasi Spinner

Kita menyiapkan daftar persentase tip dalam bentuk list (tipOptions), lalu memasangnya ke spinner menggunakan ArrayAdapter. Dengan begitu, spinner akan menampilkan pilihan seperti 10%, 15%, dst.

4. Fungsi updateTip()

Fungsi ini adalah inti dari logika kalkulasi. Ia membaca nilai biaya, persentase tip yang dipilih dari spinner, dan status dari switch. Kemudian ia menghitung tip dan memperbarui teks di TextView.

5. Event Listener

Agar aplikasi merespons secara real-time, kita menambahkan listener:

- Pada EditText, kita menggunakan TextWatcher agar setiap kali pengguna mengetik, tip langsung dihitung ulang.
- Pada Switch, kita mendengarkan perubahan status.
- Pada Spinner, kita merespons ketika pengguna memilih persentase baru.

Semua listener tersebut akan memanggil fungsi updateTip() agar tampilan hasil selalu sesuai input.

Perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi

XML + View System

XML menggunakan pendekatan deklaratif statis di mana elemen UI ditulis dalam file XML terpisah dan dikaitkan dengan logika di file Kotlin/Java melalui kode imperative. Ini adalah pendekatan konvensional dan telah digunakan sejak awal Android.

Kelebihan:

- Sudah matang dan stabil, digunakan sejak Android awal.
- Lebih familiar bagi developer lama.
- Banyak dokumentasi, tutorial, dan komunitas luas.
- Terpisah antara desain dan logika (bisa didesain pakai layout editor).

Kekurangan:

- Perlu findViewById() atau ViewBinding → verbose.
- Sulit untuk membuat UI yang dinamis atau reaktif.
- Penambahan fitur atau animasi lebih kompleks.
- Struktur hierarki UI bisa rumit dan lambat saat render.

Jetpack Compose

Jetpack Compose adalah framework UI declarative modern berbasis Kotlin, yang memungkinkan kita membangun antarmuka UI secara langsung di dalam file Kotlin, menyatukan logika dan tampilan dalam satu tempat. Komponen UI di Compose bisa langsung merefleksikan perubahan data secara otomatis (reactive).

Kelebihan:

- Lebih modern, efisien, dan ringkas.

- UI dan logika digabung → mudah di-maintain.
- Reaktif → otomatis merender ulang saat state berubah.
- Mendukung tema dinamis, animasi, dan custom UI dengan lebih mudah.
- Cocok untuk arsitektur modern (MVVM, Clean Architecture).

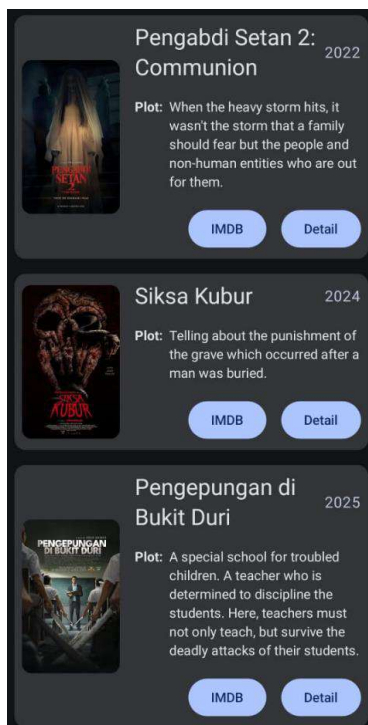
Kekurangan:

- Butuh penyesuaian untuk developer lama.
- Beberapa komponen UI belum sekomplit XML (tapi terus berkembang).
- Dokumentasi dan tooling masih dalam tahap berkembang.
- Kurang cocok untuk proyek Android lama (pre-Jetpack).

MODUL 3: BUILD A SCROLLABLE LIST

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
 8. Aplikasi berbasis XML harus menggunakan ViewBinding
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?



Gambar 3.1 1 Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 3.1 2 Contoh UI Detail

A. Source Code menggunakan Jetpack Compose MainActivity.kt

```

1 package com.example.scrollablelist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import com.example.scrollablelist.navigation.NavGraph
7 import com.example.scrollablelist.ui.theme.ScrollableListTheme
8
9 class MainActivity : ComponentActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
11         super.onCreate(savedInstanceState)
12         setContent {
13             ScrollableListTheme {
14                 NavGraph()
15             }
16         }
17     }
18 }

```

Tabel 3.1 1 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 3

B. Source Code menggunakan XML

MainActivity.kt

```
1 package com.example.scrollablelist2
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import
6 com.example.scrollablelist2.databinding.ActivityMainBinding
7
8 class MainActivity : AppCompatActivity() {
9     private lateinit var binding: ActivityMainBinding
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         binding = ActivityMainBinding.inflate(layoutInflater)
14         setContentView(binding.root)
15     }
16 }
```

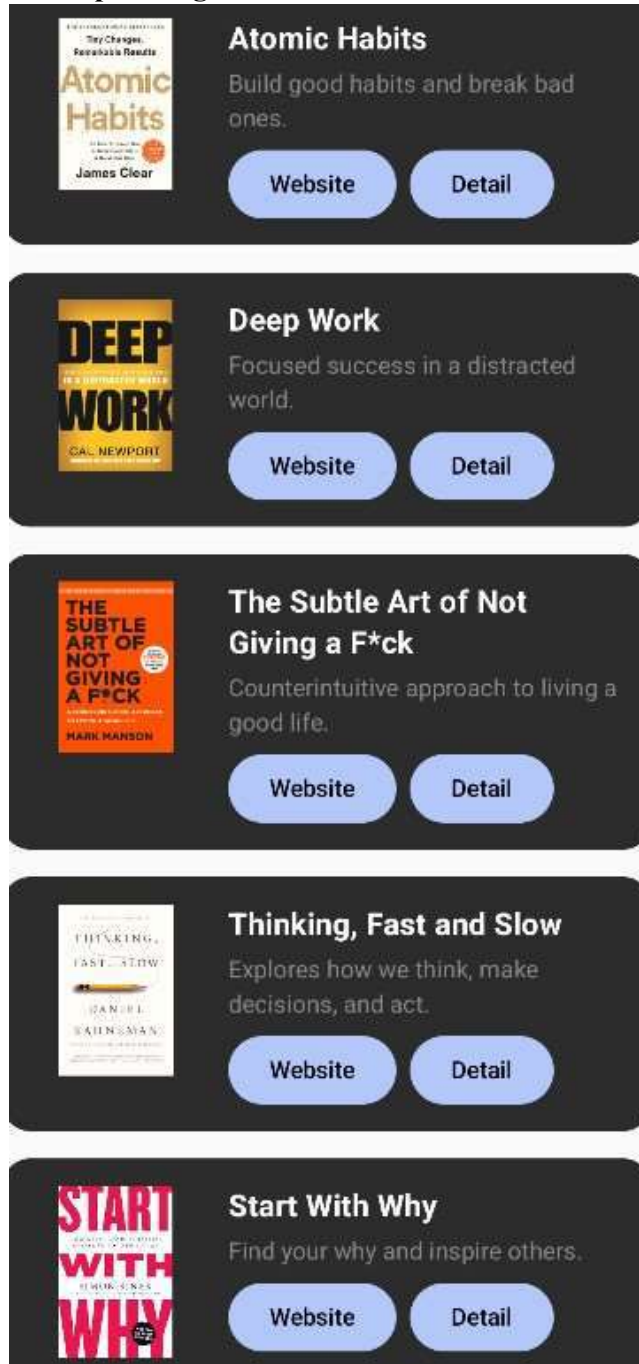
Tabel 3.1 2 Source Code Jawaban Soal 1 MainActivity.kt Jetpack Compose Modul 3

activity_main.xml

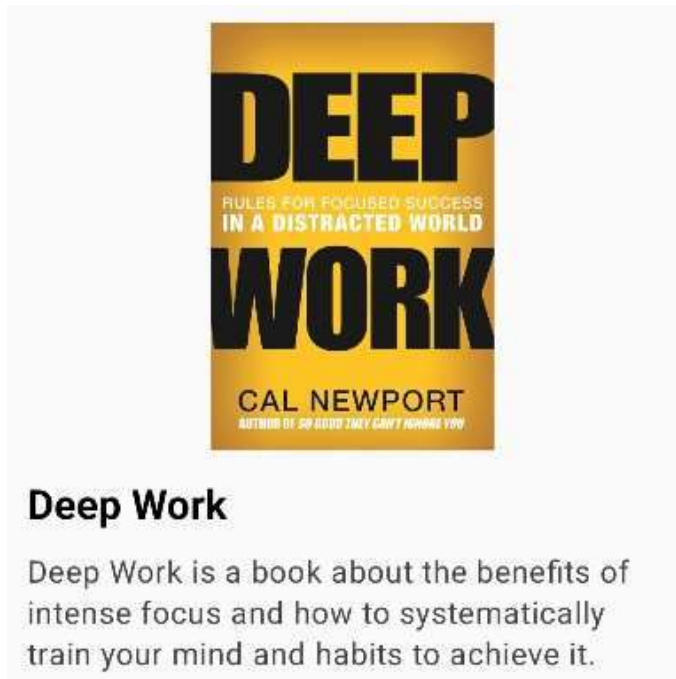
```
1 <?xml version="1.0" encoding="utf-8"?>
2     <androidx.constraintlayout.widget.ConstraintLayout
3
4     xmlns:android="http://schemas.android.com/apk/res/android"
5     xmlns:app="http://schemas.android.com/apk/res-auto"
6     xmlns:tools="http://schemas.android.com/tools"
7     android:id="@+id/mainLayout"
8     android:layout_width="match_parent"
9     android:layout_height="match_parent"
10    tools:context=".MainActivity">
11
12    <!-- Tempat untuk menampilkan fragment -->
13    <androidx.fragment.app.FragmentContainerView
14        android:id="@+id/nav_host_fragment"
15
16        android:name="androidx.navigation.fragment.NavHostFragment"
17        android:layout_width="0dp"
18        android:layout_height="0dp"
19        app:defaultNavHost="true"
20        app:navGraph="@navigation/nav_graph"
21        app:layout_constraintTop_toTopOf="parent"
22        app:layout_constraintBottom_toBottomOf="parent"
23        app:layout_constraintStart_toStartOf="parent"
24        app:layout_constraintEnd_toEndOf="parent" />
25    </androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 3.1 3 Source Code Jawaban Soal 1 activity_maint.XML Modul 3

C. Output Program



Gambar 3.1 3 Screenshot Tampilan UI List



Gambar 3.1 4 Screenshot Tampilan UI Detail

D. Pembahasan

MainActivity.kt Jetpack Compose:

- ComponentActivity digunakan karena Jetpack Compose tidak memerlukan AppCompatActivity.
- setContent { ... } adalah titik masuk untuk Compose UI.
- NavGraph() adalah fungsi Compose yang mengatur navigasi antar layar (HomeScreen, DetailScreen).
- Tidak ada layout XML: Semua UI dibangun langsung dengan fungsi-fungsi Compose (@Composable).
- Tema (ScrollableListTheme) diterapkan langsung lewat kode.

MainActivity.kt XML:

- AppCompatActivity digunakan karena UI berbasis pada XML dan ViewBinding atau findViewById.
- setContentView(...) memuat layout dari file XML (activity_main.xml).
- Navigasi dan fragment biasanya dikelola melalui NavHostFragment, FragmentManager, dan NavController.

Alasan RecyclerView Masih Digunakan

1. Kompatibilitas dan Basis Kode Lama (Legacy Projects)

Banyak aplikasi besar dan perusahaan sudah lama menggunakan RecyclerView, dan mereka tidak bisa langsung memigrasi ke Compose karena:

- Ukuran proyek yang besar
- Risiko tinggi perubahan UI/UX
- Tim belum semua menguasai Compose

2. Ekosistem yang Sudah Matang

RecyclerView punya:

- Dukungan kuat dari berbagai pustaka eksternal
- Banyak contoh, dokumentasi, Stack Overflow support
- Komponen tambahan seperti ItemTouchHelper, DiffUtil, dll.

3. Kontrol Lebih Rinci

Dengan RecyclerView.Adapter, kamu bisa:

- Mengelola jenis tampilan berbeda (multiple view types)
- Menyesuaikan performa scrolling dengan granularitas tinggi
- Menggunakan fitur seperti prefetching, caching, animasi kompleks

4. Stabilitas Produksi

RecyclerView telah teruji di banyak aplikasi produksi selama bertahun-tahun, sehingga dianggap stabil dan aman.

5. Beberapa Developer Belum Pindah ke Compose

Compose masih terbilang baru (stabil sejak 2021). Adopsi Compose masih berlangsung secara bertahap:

- Sebagian tim masih menggunakan View Binding/XML
- Butuh pelatihan ulang dan refactor besar

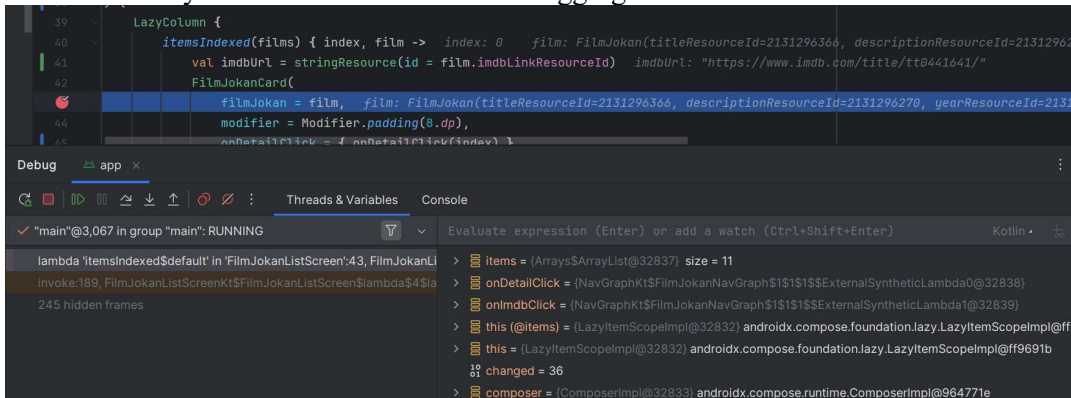
Kenapa LazyColumn Tetap Lebih Baik untuk Proyek Baru?

- Kode lebih pendek, bersih, dan deklaratif
- Tidak perlu ViewHolder, Adapter, notifyDataSetChanged, dsb
- Lebih mudah mengimplementasikan animasi dan state
- Integrasi erat dengan Compose UI lainnya

MODUL 4: VIEWMODEL AND DEBUGGING

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
 - c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
 - d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 4.1 1 Contoh Penggunaan Debugger

A. Source Code menggunakan Jetpack Compose MainActivity.kt

```
1 package com.example.scrollablelist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import com.example.scrollablelist.navigation.NavGraph
7 import com.example.scrollablelist.ui.theme.ScrollableListTheme
8
9 class MainActivity : ComponentActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
```

11	super.onCreate(savedInstanceState)
12	setContent {
13	ScrollableListTheme {
14	NavGraph()
15	}
16	}
17	}
18	}

Tabel 4.1 1 Source Code MainActivity.kt Jetpack Compose Modul 4

Book.kt

1	package com.example.scrollablelist.model
2	
3	data class Book(
4	val id: Int,
5	val title: String,
6	val imageUrl: String,
7	val webUrl: String,
8	val shortDescription: String,
9	val fullDescription: String
10	)

Tabel 4.1 2 Source Code Book.kt Jetpack Compose

BookViewModel.kt

1	package com.example.scrollablelist.viewmodel
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	import androidx.lifecycle.viewModelScope
7	import com.example.scrollablelist.model.Book
8	import kotlinx.coroutines.flow.MutableStateFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.launch
11	import timber.log.Timber
12	
13	class BookViewModel(private val query: String) : ViewModel() {
14	
15	private val _bookList =
16	MutableStateFlow<List<Book>>(emptyList())
17	val bookList: StateFlow<List<Book>> = _bookList
18	
19	private val _event = MutableStateFlow<Event?>(null)
20	val event: StateFlow<Event?> = _event
21	
22	init {
23	Timber.d("ViewModel initialized with query: \$query")
24	loadBooks(query)
25	}
26	
27	private fun loadBooks(query: String) {
28	viewModelScope.launch {
29	

```

30 // Simulasi mengambil data dari sumber data
31 (misalnya database, API, dll)
32 val books = listOf(
33     Book(
34         1,
35         "Atomic Habits",
36         "https://m.media-
37 amazon.com/images/I/91bYsX41DVL.jpg",
38
39         "https://www.goodreads.com/book/show/40121378-atomic-habits",
40         "Build good habits and break bad ones.",
41         "Atomic Habits by James Clear offers a
42 proven framework for improving everyday life. Learn how tiny
43 changes add up to remarkable results."
44     ),
45     Book(
46         2,
47         "Deep Work",
48         "https://m.media-
49 amazon.com/images/I/91nujEwIpYL._SL1500_.jpg",
50
51         "https://www.goodreads.com/book/show/25744928-deep-work",
52         "Focused success in a distracted world.",
53         "Deep Work is a book about the benefits of
54 intense focus and how to systematically train your mind and
55 habits to achieve it."
56     ),
57     Book(
58         3,
59         "The Subtle Art of Not Giving a F*ck",
60         "https://m.media-
61 amazon.com/images/I/71QKQ9mwV7L.jpg",
62
63         "https://www.goodreads.com/book/show/28257707-the-subtle-art-
64 of-not-giving-a-f-ck",
65         "Counterintuitive approach to living a good
66 life.",
67         "Mark Manson's book shows that life's
68 struggles give it meaning, and teaches how to stop trying to be
69 positive all the time."
70     ),
71     Book(
72         4,
73         "Thinking, Fast and Slow",
74         "https://m.media-
75 amazon.com/images/I/61fdrEuPJwL._SL1500_.jpg",
76
77         "https://www.goodreads.com/book/show/11468377-thinking-fast-
78 and-slow",
79         "Explores how we think, make decisions, and
80 act.",
81         "Nobel laureate Daniel Kahneman explores
82 two modes of thought: 'fast', intuitive thinking, and 'slow',
83 deliberate thinking."

```

```

84         ),
85         Book(
86             5,
87             "Start With Why",
88             "https://m.media-
89 amazon.com/images/I/71NBZIExBCL._SY466_.jpg",
90
91             "https://www.goodreads.com/book/show/7108725-start-with-why",
92             "Find your why and inspire others.",
93             "Simon Sinek explains how leaders can
94 inspire cooperation, trust and change by focusing on the WHY
95 behind their mission."
96         ),
97         Book(
98             6,
99             "The Power of Now",
100            "https://m.media-
101 amazon.com/images/I/61Ij8nLooNL._SL1500_.jpg",
102            "https://www.amazon.com/Power-Now-Guide-
103 Spiritual-Enlightenment/dp/1577314808",
104            "A guide to spiritual enlightenment.",
105            "Eckhart Tolle's guide helps readers
106 discover the importance of living in the present moment and
107 letting go of the past and future."
108         ),
109         Book(
110             7,
111             "Can't Hurt Me",
112             "https://m.media-
113 amazon.com/images/I/81gTRv2HXrL._SL1500_.jpg",
114
115             "https://www.goodreads.com/book/show/41721428-can-t-hurt-me",
116             "Master your mind and defy the odds.",
117             "David Goggins shares his incredible life
118 story and teaches readers how to overcome pain, fear, and self-
119 doubt to reach their full potential."
120         ),
121         Book(
122             8,
123             "Educated",
124             "https://m.media-
125 amazon.com/images/I/81WojUxbFL.jpg",
126
127             "https://www.goodreads.com/book/show/35133922-educated",
128             "A memoir of transformation through
129 education.",
130             "Tara Westover tells her story of growing
131 up in a strict and abusive household in rural Idaho and how she
132 escaped through learning and education."
133         )
134     )
135 )
136
137 // Log saat data masuk ke dalam list
138 Timber.d("Books loaded: $books")

```

```

139         _bookList.value = books
140     }
141 }
142
143 fun onItemClick(book: Book) {
144     // Log saat detail tombol ditekan
145     Timber.d("Detail button clicked for book:
146 ${book.title}")
147     _event.value = Event.NavigateToDetail(book)
148 }
149
150 fun onWebButtonClicked(book: Book) {
151     // Log saat tombol Explicit Intent ditekan
152     Timber.d("Web button clicked for book: ${book.title}")
153     _event.value = Event.OpenWebUrl(book.webUrl)
154 }
155
156 sealed class Event {
157     data class NavigateToDetail(val book: Book) : Event()
158     data class OpenWebUrl(val url: String) : Event()
159 }
160 }
161
162 class BookViewModelFactory(private val query: String) :
163 ViewModelProvider.Factory {
164     override fun <T : ViewModel> create(modelClass: Class<T>):
165 T {
166         if
167 (modelClass.isAssignableFrom(BookViewModel::class.java)) {
168             return BookViewModel(query) as T
169         }
170         throw IllegalArgumentException("Unknown ViewModel
171 class")
172     }
173 }

```

Tabel 4.1 3 Source Code BookViewModel.kt Jetpack Compose

DetailScreen.kt

```

1 package com.example.scrollablelist.screens
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.shape.RoundedCornerShape
6 import androidx.compose.material3.*
7 import androidx.compose.runtime.Composable
8 import androidx.compose.ui.Modifier
9 import androidx.compose.ui.draw.clip
10 import androidx.compose.ui.graphics.Color
11 import androidx.compose.ui.text.font.FontWeight
12 import androidx.compose.ui.unit.dp
13 import androidx.compose.ui.unit.sp
14 import coil.compose.rememberAsyncImagePainter
15
16 @Composable

```

```

17 fun DetailScreen(title: String, imageUrl: String,
18 fullDescription: String) {
19     Column(
20         modifier = Modifier
21             .fillMaxSize()
22             .padding(16.dp)
23     ) {
24         Image(
25             painter = rememberAsyncImagePainter(imageUrl),
26             contentDescription = title,
27             modifier = Modifier
28                 .fillMaxWidth()
29                 .height(250.dp)
30                 .clip(RoundedCornerShape(16.dp))
31         )
32
33         Spacer(modifier = Modifier.height(16.dp))
34
35         Text(
36             text = title,
37             style = MaterialTheme.typography.headlineSmall,
38             fontWeight = FontWeight.Bold,
39             fontSize = 24.sp,
40             color = Color.Black
41         )
42
43         Spacer(modifier = Modifier.height(12.dp))
44
45         Text(
46             text = fullDescription,
47             style = MaterialTheme.typography.bodyLarge,
48             fontSize = 18.sp,
49             color = Color.DarkGray
50         )
51     }
52 }

```

Tabel 4.1 4 Source Code DetailScreen.kt Jetpack Compose

HomeScreen.kt

```

1 package com.example.scrollablelist.screens
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.lazy.LazyColumn
8 import androidx.compose.foundation.lazy.items
9 import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.*
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.draw.clip
15 import androidx.compose.ui.graphics.Color

```

```

16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19 import androidx.lifecycle.viewmodel.compose.viewModel
20 import androidx.navigation.NavController
21 import com.example.scrollablelist.model.Book
22 import com.example.scrollablelist.viewmodel.BookViewModel
23 import
24 com.example.scrollablelist.viewmodel.BookViewModelFactory
25 import coil.compose.rememberAsyncImagePainter
26 import timber.log.Timber
27
28 @Composable
29 fun HomeScreen(navController: NavController) {
30     val viewModel: BookViewModel = viewModel(factory =
31     BookViewModelFactory("some query"))
32     val books by viewModel.bookList.collectAsState()
33     val event by viewModel.event.collectAsState()
34
35     LaunchedEffect(event) {
36         when (event) {
37             is BookViewModel.Event.NavigateToDetail -> {
38                 val book = (event as
39                 BookViewModel.Event.NavigateToDetail).book
40
41                 navController.navigate("detail/${Uri.encode(book.title)}/${Ur
42                 i.encode(book.imageUrl)}/${Uri.encode(book.fullDescription)}"
43                 )
44             }
45             is BookViewModel.Event.OpenWebUrl -> {
46                 val url = (event as
47                 BookViewModel.Event.OpenWebUrl).url
48                 val intent = Intent(Intent.ACTION_VIEW,
49                 Uri.parse(url))
50                 navController.context.startActivity(intent)
51             }
52             null -> Unit
53         }
54     }
55
56     LazyColumn(
57         contentPadding = PaddingValues(16.dp),
58         verticalArrangement = Arrangement.spacedBy(16.dp)
59     ) {
60         items(books) { book ->
61             Card(
62                 shape = RoundedCornerShape(16.dp),
63                 colors
64                 CardDefaults.cardColors(containerColor = Color(0xFF2C2C2C)),
65                 modifier = Modifier.fillMaxWidth()
66             ) {
67                 Row(
68                     modifier = Modifier
69                     .padding(16.dp)

```

```

70         .fillMaxWidth()
71     ) {
72         Image(
73             painter
74 rememberAsyncImagePainter(book.imageUrl),
75             contentDescription = book.title,
76             modifier = Modifier
77                 .size(100.dp)
78                 .clip(RoundedCornerShape(16.dp))
79         )
80
81         Spacer(modifier = Modifier.width(16.dp))
82
83         Column(
84             modifier = Modifier.weight(1f)
85         ) {
86             Text(
87                 text = book.title,
88                 style
89 MaterialTheme.typography.titleMedium,
90                 color = Color.White,
91                 fontWeight = FontWeight.Bold,
92                 fontSize = 18.sp
93             )
94             Spacer(modifier
95 Modifier.height(4.dp))
96             Text(
97                 text = book.shortDescription,
98                 style
99 MaterialTheme.typography.bodyMedium,
100                 color = Color.Gray,
101                 maxLines = 2
102             )
103
104             Spacer(modifier
105 Modifier.height(8.dp))
106
107             Row {
108                 Button(
109                     onClick = {
110 viewModel.onWebButtonClicked(book)
111                     },
112                     colors
113 ButtonDefaults.buttonColors(
114                         containerColor
115 Color(0xFFB3C7F9),
116                         contentColor = Color.Black
117                     ),
118                     shape
119 RoundedCornerShape(50),
120                     modifier
121 Modifier.height(40.dp)
122                 ) {
123
124

```



```

125         Text(text = "Website")
126     }
127
128     Spacer(modifier =
129 Modifier.width(8.dp) )
130
131     Button(
132         onClick = {
133
134         viewModel.onItemClicked(book)
135     },
136     colors =
137 ButtonDefaults.buttonColors(
138         containerColor =
139 Color(0xFFB3C7F9),
140         contentColor = Color.Black
141     ),
142     shape =
143 RoundedCornerShape(50),
144     modifier =
145 Modifier.height(40.dp)
146 ) {
147     Text(text = "Detail")
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }

```

Tabel 4.1 5 Source Code HomeScreen.kt Jetpack Compose

MyApplication.kt

```

1 package com.example.scrollablelist
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MyApplication : Application() {
7     override fun onCreate() {
8         super.onCreate()
9         Timber.plant(Timber.DebugTree())
10    }
11 }

```

Tabel 4.1 6 Source Code MyApplication.kt Jetpack Compose

Navigation.kt

```

1 package com.example.scrollablelist.navigation
2
3 import androidx.compose.runtime.Composable
4 import androidx.navigation.NavHostController
5 import androidx.navigation.compose.NavHost

```

6	import androidx.navigation.compose.composable	
7	import androidx.navigation.compose.rememberNavController	
8	import com.example.scrollablelist.screens.DetailScreen	
9	import com.example.scrollablelist.screens.HomeScreen	
10		
11	@Composable	
12	fun NavGraph(navController: NavController)	=
13	rememberNavController()) {	
14	NavHost(	
15	navController = navController,	
16	startDestination = "home"	
17	) {	
18	composable("home") {	
19	HomeScreen(navController)	
20	}	
21		
22	composable("detail/{title}/{imageUrl}/{fullDescription}")	{
23	backStackEntry ->	
24	val title	=
25	backStackEntry.arguments?.getString("title") ?: ""	
26	val imageUrl	=
27	backStackEntry.arguments?.getString("imageUrl") ?: ""	
28	val fullDescription	=
29	backStackEntry.arguments?.getString("fullDescription") ?: ""	
30	DetailScreen(title, imageUrl, fullDescription)	
31	}	
32	}	
33	}	

Tabel 4.1 7 Source Code Navigation.kt Jetpack Compose

B. Source Code menggunakan XML

MainActivity.kt

1	package com.example.scrollablelist2
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
6	com.example.scrollablelist2.databinding.ActivityMainBinding
7	
8	class MainActivity: AppCompatActivity() {
9	private lateinit var binding: ActivityMainBinding
10	
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	binding = ActivityMainBinding.inflate(layoutInflater)
14	setContentView(binding.root)
15	}
16	}

Tabel 4.1 8 Source Code MainActivity.kt XML Modul 4

Book.kt

1	package com.example.scrollablelist2.data
2	

3	data class Book(
4	val id: Int,
5	val title: String,
6	val imageUrl: String,
7	val webUrl: String,
8	val shortDescription: String,
9	val fullDescription: String
10	)

Tabel 4.1 9 Source Code Book.kt XML

BookAdapter.kt

1	package com.example.scrollablelist2.ui
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.bumptech.glide.Glide
7	import com.example.scrollablelist2.data.Book
8	import com.example.scrollablelist2.databinding.ItemBookBinding
9	
10	class BookAdapter(private var bookList: List<Book>, private val
11	onBookClicked: (Book) -> Unit) :
12	RecyclerView.Adapter<BookAdapter.BookViewHolder>() {
13	
14	inner class BookViewHolder(val binding: ItemBookBinding) :
15	RecyclerView.ViewHolder(binding.root)
16	
17	override fun onCreateViewHolder(parent: ViewGroup, viewType:
18	Int): BookViewHolder {
19	val binding =
20	ItemBookBinding.inflate(LayoutInflater.from(parent.context),
21	parent, false)
22	return BookViewHolder(binding)
23	}
24	
25	override fun onBindViewHolder(holder: BookViewHolder,
26	position: Int) {
27	val book = bookList[position]
28	holder.binding.apply {
29	titleTextView.text = book.title
30	descTextView.text = book.shortDescription
31	
32	Glide.with(imageView.context).load(book.imageUrl).into(imageView)
33	
34	root.setOnClickListener {
35	onBookClicked(book)
36	}
37	}
38	}
39	
40	override fun getItemCount(): Int = bookList.size
41	
42	fun updateBooks(newBooks: List<Book>) {
43	bookList = newBooks

44	notifyDataSetChanged()
45	}
46	}

Tabel 4.1 10 Source Code BookAdapter.kt XML

BookDetailFragment.kt

1	package com.example.scrollablelist2.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import com.bumptech.glide.Glide
9	import
10	com.example.scrollablelist2.databinding.FragmentBookDetailBinding
11	
12	class BookDetailFragment : Fragment() {
13	
14	private lateinit var binding: FragmentBookDetailBinding
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container: ViewGroup?,
18	savedInstanceState: Bundle?
19	): View? {
20	binding = FragmentBookDetailBinding.inflate(inflater,
21	container, false)
22	
23	val args =
24	BookDetailFragmentArgs.fromBundle(requireArguments())
25	binding.detailTitle.text = args.title
26	binding.detailDescription.text = args.description
27	
28	Glide.with(this)
29	.load(args.imageUrl)
30	.into(binding.detailImageView)
31	
32	return binding.root
33	}
34	}

Tabel 4.1 11 Source Code BookDetailFragment.kt XML

BookListFragment.kt

1	package com.example.scrollablelist2.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.LifecycleScope
10	

```

11 import
12 com.example.scrollablelist2.databinding.FragmentBookListBinding
13 import kotlinx.coroutines.flow.collect
14 import timber.log.Timber
15
16 class BookListFragment : Fragment() {
17
18     private lateinit var binding: FragmentBookListBinding
19     private val viewModel: BookViewModel by activityViewModels
20 { BookViewModelFactory() }
21
22     override fun onCreateView(inflater: LayoutInflater,
23 container: ViewGroup?, savedInstanceState: Bundle?): View? {
24         binding = FragmentBookListBinding.inflate(inflater,
25 container, false)
26
27         val adapter = BookAdapter(emptyList()) { book ->
28             viewModel.onBookSelected(book)
29         }
30         binding.bookRecyclerView.adapter = adapter
31
32         // Mengamati perubahan daftar buku
33         lifecycleScope.launchWhenStarted {
34             viewModel.books.collect { books ->
35                 Timber.d("Books data collected: ${books.size}")
36                 adapter.updateBooks(books)
37             }
38         }
39
40         return binding.root
41     }
42 }

```

Tabel 4.1 12 Source Code BookListFragment.kt XML

BookViewModel.kt

```

1 package com.example.scrollablelist2.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import kotlinx.coroutines.flow.MutableStateFlow
6 import kotlinx.coroutines.flow.StateFlow
7 import kotlinx.coroutines.launch
8 import com.example.scrollablelist2.data.Book
9 import timber.log.Timber
10
11 class BookViewModel : ViewModel() {
12
13     private val _books =
14     MutableStateFlow<List<Book>>(emptyList())
15     val books: StateFlow<List<Book>> = _books
16
17     init {
18         fetchBooks()
19     }
20 }

```

```

20
21     private fun fetchBooks() {
22         // Log ketika data buku di-load
23         Timber.d("Loading books data")
24
25         // Simulasi mengambil data buku
26         val bookList = listOf(
27             Book(1, "Atomic Habits",
28                 "https://m.media-
29 amazon.com/images/I/91bYsX41DVL.jpg",
30                 "https://www.goodreads.com/book/show/40121378-
31 atomic-habits",
32                 "Build good habits and break bad ones.",
33                 "Atomic Habits by James Clear offers a proven
34 framework for improving everyday life. Learn how tiny changes
35 add up to remarkable results."),
36
37             Book(2, "Deep Work",
38                 "https://m.media-
39 amazon.com/images/I/91nujEWIpYL._SL1500_.jpg",
40                 "https://www.goodreads.com/book/show/25744928-
41 deep-work",
42                 "Focused success in a distracted world.",
43                 "Deep Work is a book about the benefits of
44 intense focus and how to systematically train your mind and
45 habits to achieve it."),
46
47             Book(3, "The Subtle Art of Not Giving a F*ck",
48                 "https://m.media-
49 amazon.com/images/I/71QKQ9mwV7L.jpg",
50                 "https://www.goodreads.com/book/show/28257707-
51 the-subtle-art-of-not-giving-a-f-ck",
52                 "Counterintuitive approach to living a good
53 life.",
54                 "Mark Manson's book shows that life's struggles
55 give it meaning, and teaches how to stop trying to be positive
56 all the time."),
57
58             Book(4, "Thinking, Fast and Slow",
59                 "https://m.media-
60 amazon.com/images/I/61fdrEuPJwL._SL1500_.jpg",
61                 "https://www.goodreads.com/book/show/11468377-
62 thinking-fast-and-slow",
63                 "Explores how we think, make decisions, and
64 act.",
65                 "Nobel laureate Daniel Kahneman explores two
66 modes of thought: 'fast', intuitive thinking, and 'slow',
67 deliberate thinking."),
68
69             Book(5, "Start With Why",
70                 "https://m.media-
71 amazon.com/images/I/71NBZIExBCL._SY466_.jpg",
72                 "https://www.goodreads.com/book/show/7108725-
73 start-with-why",

```

74	"Find your why and inspire others.",
75	"Simon Sinek explains how leaders can inspire
76	cooperation, trust and change by focusing on the WHY behind
77	their mission."),
78	
79	Book(6, "The Power of Now",
80	"https://m.media-
81	amazon.com/images/I/61Ij8nLoonL._SL1500_.jpg",
82	"https://www.amazon.com/Power-Now-Guide-
83	Spiritual-Enlightenment/dp/1577314808",
84	"A guide to spiritual enlightenment.",
85	"Eckhart Tolle's guide helps readers discover
86	the importance of living in the present moment and letting go
87	of the past and future."),
88	
89	Book(7, "Can't Hurt Me",
90	"https://m.media-
91	amazon.com/images/I/81gTRv2HXrL._SL1500_.jpg",
92	"https://www.goodreads.com/book/show/41721428-
93	can-t-hurt-me",
94	"Master your mind and defy the odds.",
95	"David Goggins shares his incredible life story
96	and teaches readers how to overcome pain, fear, and self-doubt
97	to reach their full potential."),
98	
99	Book(8, "Educated",
100	"https://m.media-
101	amazon.com/images/I/81WojUxbFL.jpg",
102	"https://www.goodreads.com/book/show/35133922-
103	educated",
104	"A memoir of transformation through
105	education.",
106	"Tara Westover tells her story of growing up in
107	a strict and abusive household in rural Idaho and how she
108	escaped through learning and education."),
109	
110	Book(9, "Sapiens: A Brief History of Humankind",
112	"https://m.media-
113	amazon.com/images/I/91MJzUOZ3CL.jpg",
114	"https://www.goodreads.com/book/show/23692271-
115	sapiens",
116	"A journey through human history.",
117	"Yuval Noah Harari explores the history of
118	humankind from the Stone Age to the modern age, examining how
119	biology and history have defined human societies."),
120	
121	Book(10, "Becoming",
122	"https://m.media-
123	amazon.com/images/I/71kW6hbcfPL.jpg",
124	"https://www.goodreads.com/book/show/38746485-
125	becoming",
126	"The memoir of Michelle Obama.",
127	
128	

```

129         "Becoming by Michelle Obama is a powerful
130 memoir where she reflects on her personal journey from the South
131 Side of Chicago to the White House, and beyond."),
132
133         Book(11, "Educated",
134             "https://m.media-
135 amazon.com/images/I/71N9Sx5GBxL.jpg",
136             "https://www.goodreads.com/book/show/25733515-
137 educated",
138             "A memoir about a woman who sought education.",
139             "Educated is a memoir by Tara Westover,
140 detailing her upbringing in a strict and abusive household and
141 her eventual escape through education. A story of resilience
142 and perseverance.")
143     )
144
145     // Mengupdate StateFlow dengan data buku
146     viewModelScope.launch {
147         books.emit(bookList)
148         Timber.d("Books loaded: ${bookList.size} books")
149     }
150 }
151
152 // Event untuk tombol di klik
153 fun onBookSelected(book: Book) {
154     Timber.d("Book selected: ${book.title}")
155 }

```

Tabel 4.1 13 Source Code BookViewModel.kt XML

BookViewFactory.kt

```

1 package com.example.scrollablelist2.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import androidx.lifecycle.viewmodel.CreationExtras
6
7 class BookViewModelFactory : ViewModelProvider.Factory {
8
9     override fun <T : ViewModel> create(modelClass: Class<T>,
10 extras: CreationExtras): T {
11         if
12 (modelClass.isAssignableFrom(BookViewModel::class.java)) {
13             // Return instance dari BookViewModel
14             return BookViewModel() as T
15         }
16         throw IllegalArgumentException("Unknown ViewModel
17 class")
18     }
19 }

```

Tabel 4.1 14 Source Code BookViewModelFactory.kt XML

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:id="@+id/mainLayout"
7     android:layout_width="match_parent"
8     android:layout_height="match_parent"
9     tools:context=".MainActivity">
10
11     <!-- Tempat untuk menampilkan fragment -->
12     <androidx.fragment.app.FragmentContainerView
13         android:id="@+id/nav_host_fragment"
14
15         android:name="androidx.navigation.fragment.NavHostFragment"
16         android:layout_width="0dp"
17         android:layout_height="0dp"
18         app:defaultNavHost="true"
19         app:navGraph="@navigation/nav_graph"
20         app:layout_constraintTop_toTopOf="parent"
21         app:layout_constraintBottom_toBottomOf="parent"
22         app:layout_constraintStart_toStartOf="parent"
23         app:layout_constraintEnd_toEndOf="parent" />
24 </androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 4.1 15 Source Code activity_maint.XML

fragment_book_detail.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <ScrollView
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:padding="16dp">
7
8     <LinearLayout
9         android:orientation="vertical"
10        android:layout_width="match_parent"
11        android:layout_height="wrap_content">
12
13        <ImageView
14            android:id="@+id/detailImageView"
15            android:layout_width="match_parent"
16            android:layout_height="250dp"
17            android:scaleType="centerCrop"
18            android:layout_marginBottom="16dp"
19            android:background="@drawable/rounded_image" />
20
21        <TextView
22            android:id="@+id/detailTitle"
23            android:layout_width="match_parent"
24            android:layout_height="wrap_content"
25            android:textSize="20sp">
```

26	android:textStyle="bold" />
27	
28	<TextView
29	android:id="@+id/detailDescription"
30	android:layout_width="match_parent"
31	android:layout_height="wrap_content"
32	android:layout_marginTop="8dp"
33	android:textSize="16sp" />
34	
35	</LinearLayout>
36	</ScrollView>

Tabel 4.1 16 Source Code *fragment_book_detail.XML*

fragment_book_list.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent">
7	
8	<androidx.recyclerview.widget.RecyclerView
9	android:id="@+id/bookRecyclerView"
10	android:layout_width="0dp"
11	android:layout_height="0dp"
12	android:padding="8dp"
13	
14	app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
15	app:layout_constraintTop_toTopOf="parent"
16	app:layout_constraintBottom_toBottomOf="parent"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintEnd_toEndOf="parent"/>
19	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 4.1 17 Source Code *fragment_book_list.XML*

item_book.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	card_view:cardCornerRadius="16dp"
9	card_view:cardElevation="6dp"
10	card_view:cardBackgroundColor="#2C2C2C">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="horizontal"
16	android:padding="16dp">
17	

```

18     <ImageView
19         android:id="@+id/imageView"
20         android:layout_width="100dp"
21         android:layout_height="150dp"
22         android:scaleType="centerCrop"
23         android:contentDescription="@string/book_cover"
24         android:layout_marginEnd="16dp"
25         android:background="@drawable/image_background"
26         android:clipToOutline="true" />
27
28     <LinearLayout
29         android:layout_width="0dp"
30         android:layout_height="match_parent"
31         android:layout_weight="1"
32         android:orientation="vertical">
33
34         <TextView
35             android:id="@+id/titleTextView"
36             android:layout_width="wrap_content"
37             android:layout_height="wrap_content"
38             android:text="Book Title"
39             android:textColor="@android:color/white"
40             android:textSize="18sp"
41             android:textStyle="bold" />
42
43         <TextView
44             android:id="@+id/yearTextView"
45             android:layout_width="wrap_content"
46             android:layout_height="wrap_content"
47             android:text="2025"
48             android:textColor="@android:color/darker_gray"
49             android:textSize="14sp"
50             android:layout_marginBottom="8dp" />
51
52         <TextView
53             android:id="@+id/descTextView"
54             android:layout_width="wrap_content"
55             android:layout_height="wrap_content"
56             android:text="Short description goes here"
57             android:textColor="@android:color/white"
58             android:textSize="14sp"
59             android:layout_marginBottom="8dp" />
60
61         <LinearLayout
62             android:layout_width="wrap_content"
63             android:layout_height="wrap_content"
64             android:orientation="horizontal"
65             android:layout_marginTop="8dp">
66
67             <Button
68                 android:id="@+id/buttonOpenLink"
69                 android:layout_width="wrap_content"
70                 android:layout_height="36dp"
71                 android:text="Goodreads"

```

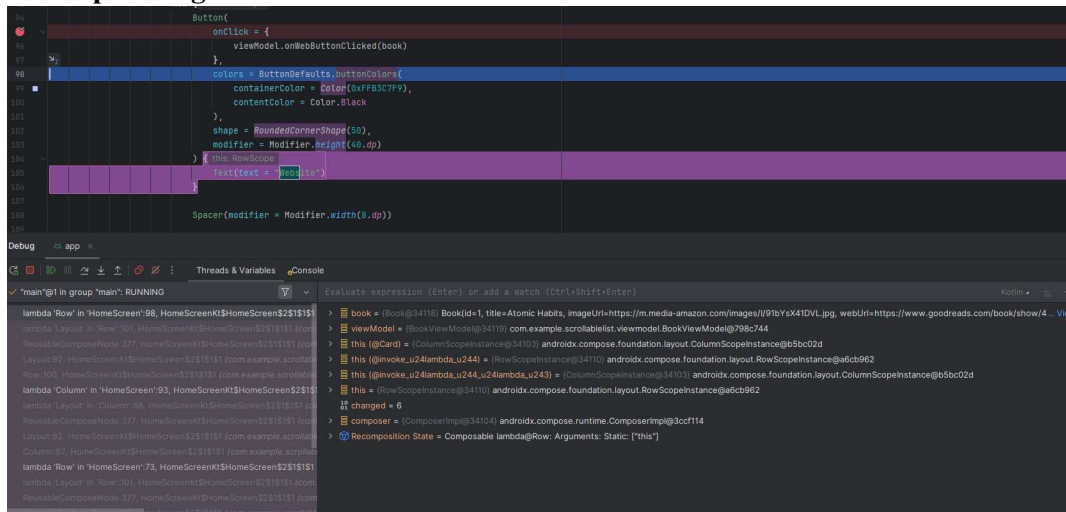
```

72         android:textColor="#000000"
73         android:backgroundTint="#B3C7F9"
74         android:textSize="14sp"
75         android:layout_marginEnd="8dp" />
76
77         <Button
78             android:id="@+id/buttonDetails"
79             android:layout_width="wrap_content"
80             android:layout_height="36dp"
81             android:text="Detail"
82             android:textColor="#000000"
83             android:backgroundTint="#B3C7F9"
84             android:textSize="14sp" />
85     </LinearLayout>
86 </LinearLayout>
87 </LinearLayout>
88 </androidx.cardview.widget.CardView>

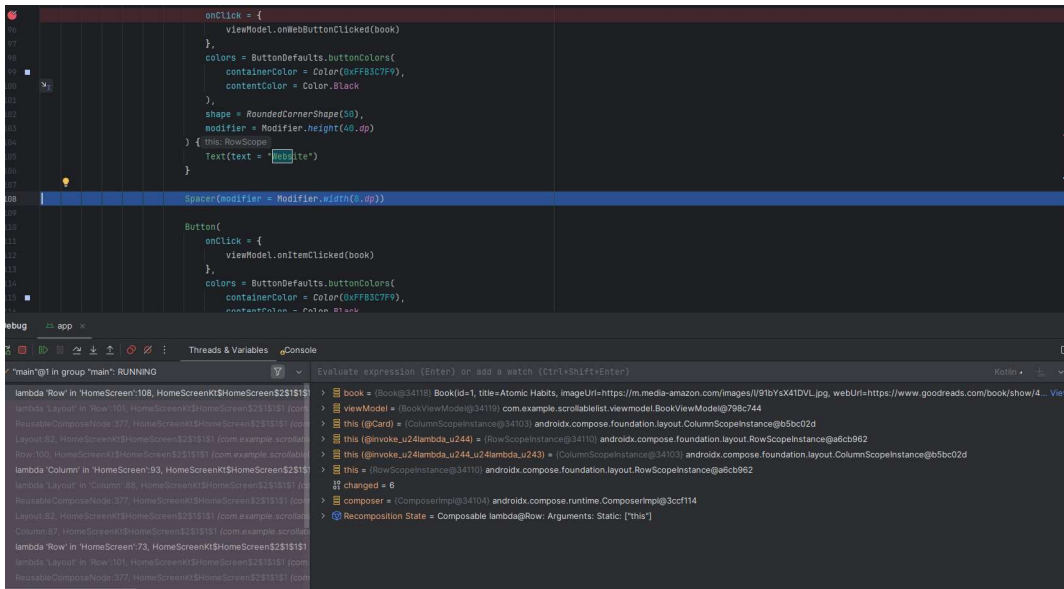
```

Tabel 4.1 18 Source Code item_book_ .XML

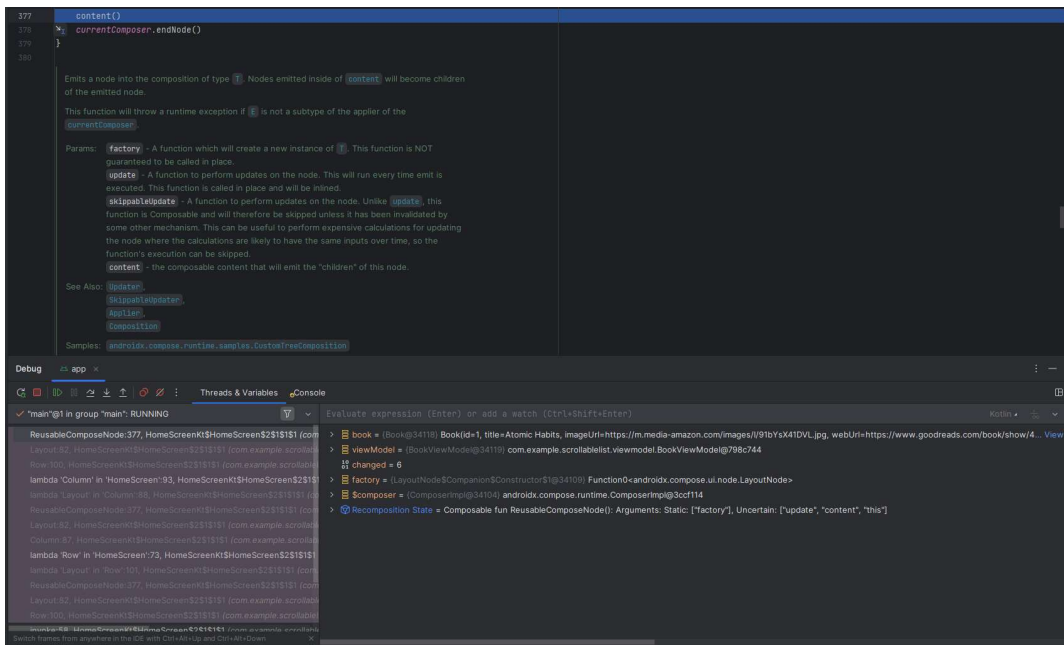
C. Output Program



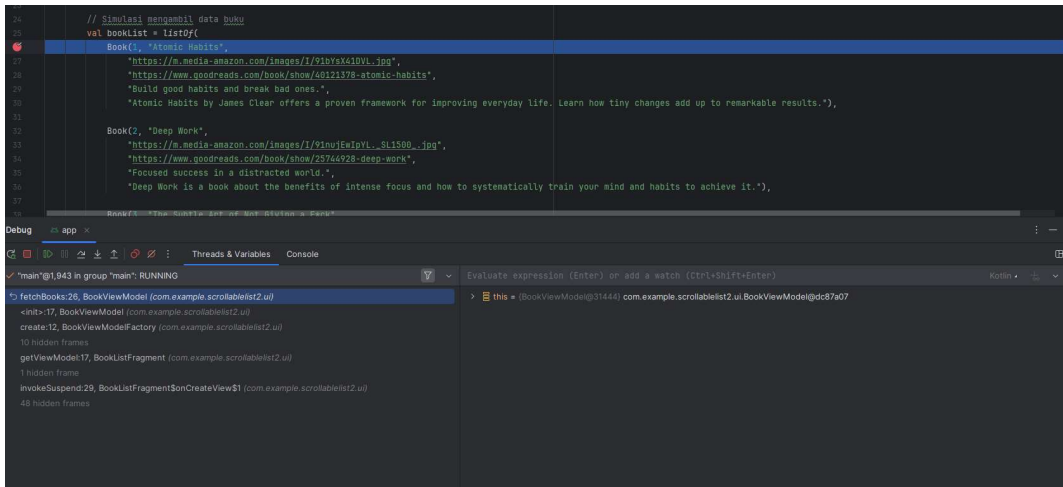
Gambar 4.1 2 Screenshot Debugging Step Into Jetpack Compose



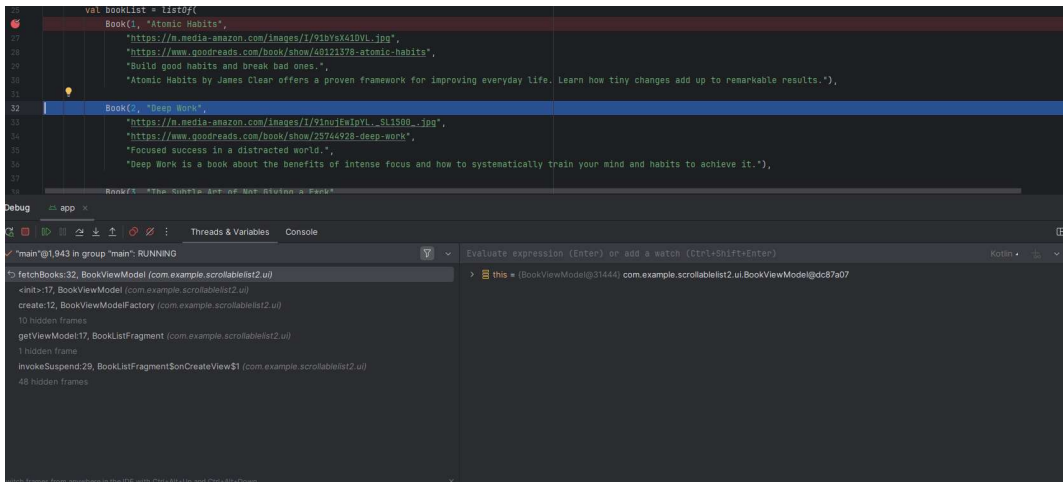
Gambar 4.1 3 Screenshot Debugging Step Over Jetpack Compose



Gambar 4.1 4 Screenshot Debugging Step Out Jetpack Compose



Gambar 4.1 5 Screenshot Debugging Step Into XML



Gambar 4.1 6 Screenshot Debugging Step Over XML

BookViewModelFactory.kt

- Custom ViewModelFactory untuk memberikan parameter (source) ke ViewModel.

HomeScreen.kt

- Menampilkan daftar Book dalam bentuk LazyColumn.
- Ketika tombol Detail diklik, memanggil onBookClick di ViewModel dan navigasi ke halaman detail.

DetailScreen.kt

- Menampilkan informasi lengkap tentang buku yang dipilih.
- Logging judul buku yang diterima ke Logcat.

MainActivity.kt XML:

- AppCompatActivity digunakan karena UI berbasis pada XML dan ViewBinding atau findViewById.
- setContentView(...) memuat layout dari file XML (activity_main.xml).
- Navigasi dan fragment biasanya dikelola melalui NavHostFragment, FragmentManager, dan NavController.

Book.kt

- Model data buku yang digunakan di seluruh aplikasi.

BookViewModel.kt

- Menyediakan data buku secara reaktif (melalui StateFlow) untuk digunakan oleh UI.

BookViewModelFactory.kt

- Alas Digunakan jika ingin membuat BookViewModel secara manual (misal inject dependency).
- Menyesuaikan dengan API baru ViewModelProvider.Factory.

BookAdapter.kt

- Menampilkan setiap buku di dalam RecyclerView.
- Menangani klik tombol untuk membuka link atau navigasi ke detail.

BookListFragment.kt

- Mengamati StateFlow dari BookViewModel.
- Menampilkan daftar buku di RecyclerView.

BookDetailFragment.kt

- Menampilkan detail lengkap buku berdasarkan argumen navigasi dari BookListFragment.

activity_main.xml

- FragmentContainerView: tempat menampilkan fragment.
- app:navGraph: menunjuk ke file navigasi (res/navigation/nav_graph.xml).
- defaultNavHost: agar tombol back sistem bisa bekerja otomatis.

fragment_book_list.xml

- RecyclerView: menampilkan daftar buku.
- Digunakan oleh BookListFragment untuk mengisi dengan BookAdapter.

item_book.xml

- ImageView: menampilkan gambar buku.
- TextView: judul dan deskripsi singkat.
- Dua tombol: buka link dan lihat detail.
- Layout ini digunakan oleh BookAdapter.

fragment_book_detail.xml

- ScrollView: agar konten bisa digulir.
- Menampilkan judul buku, gambar, dan deskripsi lengkap.
- Data diisi dari BookDetailFragmentArgs.

Apa itu Application class?

Application adalah kelas dasar di Android yang digunakan untuk menyimpan dan mengelola status global aplikasi. Android membuat instance dari class ini sebelum aktivitas, service, atau receiver pertama kali dijalankan.

Fungsi dan Kegunaan :

1. **Inisialisasi Global** : Misalnya Timber, Hilt, Firebase, atau Analytics hanya perlu diinisialisasi sekali di seluruh aplikasi.
 2. **Menyediakan Dependency Global** : Bisa digunakan sebagai semacam service locator sederhana untuk menyediakan object atau config global.
 3. **Melacak Lifecycle Global** : User bisa menambahkan ActivityLifecycleCallbacks untuk mengetahui aktivitas apa yang aktif, masuk background, dll.
- Mocking/Test Support** : Di testing, user bisa mengganti Application class untuk menyuntikkan dependency yang berbeda.

MODUL 5: CONNECT TO THE INTERNET

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
 - b. Gunakan KotlinX Serialization sebagai library JSON.
 - c. Gunakan library seperti Coil atau Glide untuk image loading.
 - d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
 - e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
 - f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
 - g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code menggunakan Jetpack Compose

MainActivity.kt

```
1 package com.example.scrollablelist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import com.example.scrollablelist.navigation.NavGraph
7 import com.example.scrollablelist.ui.theme.ScrollableListTheme
8
9 class MainActivity : ComponentActivity() {
10
11     private val apiKey = "9fad0dc9a0338ecf00596875e4cf5645" //
12     Masukkan API Key di sini
13
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16         setContent {
17             ScrollableListTheme {
18                 NavGraph()
19             }
20         }
21     }
22 }
```

Tabel 5.1.1 Source Code MainActivity.kt Jetpack Compose Modul 5

ApiService.kt

```
1 package com.example.scrollablelist.data.remote
2
3 import com.example.scrollablelist.model.MovieResponse
4 import retrofit2.http.GET
5 import retrofit2.http.Query
6
7 interface ApiService {
8     @GET("movie/popular")
9     suspend fun getPopularMovies(
10         @Query("api_key") apiKey: String
11     ): MovieResponse
12 }
```

Tabel 5.1 2 Source Code ApiService.kt Jetpack Compose

DetailScreen.kt

```
1 package com.example.scrollablelist.screens
2
3 import androidx.activity.compose.BackHandler
4 import androidx.compose.foundation.Image
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.shape.RoundedCornerShape
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.draw.clip
11 import androidx.compose.ui.graphics.Color
12 import androidx.compose.ui.text.font.FontWeight
13 import androidx.compose.ui.unit.dp
14 import androidx.compose.ui.unit.sp
15 import coil.compose.rememberAsyncImagePainter
16 import androidx.compose.material.icons.filled.ArrowBack
17
18 @Composable
19 fun DetailScreen(
20     title: String,
21     imageUrl: String,
22     fullDescription: String,
23     onBack: () -> Unit
24 ) {
25     val imageBaseUrl = "https://image.tmdb.org/t/p/w500"
26     val fullImageUrl = imageBaseUrl + imageUrl
27
28     Scaffold(
29         topBar = {
30             SmallTopAppBar(
31                 title = { Text(text = "Detail Film") },
32                 onBack = onBack
33             )
34         }
35     ) { padding ->
36         Column(
37             modifier = Modifier
```

```

38         .fillMaxSize()
39         .padding(padding)
40         .padding(16.dp)
41     ) {
42         Image(
43             painter
44 rememberAsyncImagePainter(fullImageUrl),
45             contentDescription = title,
46             modifier = Modifier
47                 .fillMaxWidth()
48                 .height(300.dp)
49                 .clip(RoundedCornerShape(16.dp))
50         )
51
52         Spacer(modifier = Modifier.height(16.dp))
53
54         Text(
55             text = title,
56             style = MaterialTheme.typography.headlineSmall,
57             fontWeight = FontWeight.Bold,
58             fontSize = 24.sp,
59             color = Color.Black
60         )
61
62         Spacer(modifier = Modifier.height(12.dp))
63
64         Text(
65             text = fullDescription,
66             style = MaterialTheme.typography.bodyLarge,
67             fontSize = 18.sp,
68             color = Color.DarkGray
69         )
70     }
71 }
72
73
74 @OptIn(ExperimentalMaterial3Api::class)
75 @Composable
76 fun SmallTopAppBar(
77     title: @Composable () -> Unit,
78     onBack: () -> Unit
79 ) {
80     TopAppBar(
81         title = title,
82         navigationIcon = {
83             IconButton(onClick = onBack) {
84                 Icon(
85                     imageVector
86 androidx.compose.material.icons.Icons.Default.ArrowBack,
87                     contentDescription = "Back"
88                 )
89             }
90         }
91     )

```

92	}
93	
94	
95	
96	

Tabel 5.1 3 Source Code DetailScreen.kt Jetpack Compose

Film.kt

1	package com.example.scrollablelist.model
2	
3	data class Film(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterPath: String?,
8	val backdropPath: String?,
9	val releaseDate: String?,
10	val homepage: String? = null
11	)

Tabel 5.1 4 Source Code Film.kt Jetpack Compose

FilmRepository.kt

1	package com.example.scrollablelist.data
2	
3	import com.example.scrollablelist.model.Film
4	import com.example.scrollablelist.network.TmdbApi
5	
6	class FilmRepository(
7	private val api: TmdbApi,
8	private val apiKey: String
9	) {
10	suspend fun getPopularFilms(): List<Film> {
11	val response = api.getPopularMovies(apiKey)
12	
13	return response.results.map { networkFilm ->
14	Film(
15	id = networkFilm.id,
16	title = networkFilm.title,
17	overview = networkFilm.overview,
18	posterPath = networkFilm.posterPath,
19	backdropPath = networkFilm.backdropPath,
20	releaseDate = networkFilm.releaseDate,
21	homepage = null
22	)
23	}
24	}
25	}

Tabel 5.1 5 Source Code FilmRepository.kt Jetpack Compose

FilmViewModel.kt

1	package com.example.scrollablelist.ui
2	
3	import androidx.lifecycle.ViewModel

```

4 import androidx.lifecycle.viewModelScope
5 import com.example.scrollablelist.data.FilmRepository
6 import com.example.scrollablelist.model.Film
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.asStateFlow
10 import kotlinx.coroutines.launch
11 import timber.log.Timber
12
26
27 class FilmViewModel(
28     private val repository: FilmRepository
29 ) : ViewModel() {
30
31     private val _filmList =
32     MutableStateFlow<List<Film>>(emptyList())
33     val filmList: StateFlow<List<Film>> =
34     _filmList.asStateFlow()
35
36     private val _event = MutableStateFlow<Event?>(null)
37     val event: StateFlow<Event?> = _event.asStateFlow()
38
39     init {
40         loadFilmsFromApi()
41     }
42
43     private fun loadFilmsFromApi() {
44         viewModelScope.launch {
45             try {
46                 val films = repository.getPopularFilms()
47                 _filmList.value = films as List<Film>
48             } catch (e: Exception) {
49                 Timber.e("Gagal memuat film dari API:
50 ${e.message}")
51             }
52         }
53     }
54
55     fun onItemClickClicked(film: Film) {
56         _event.value = Event.NavigateToDetail(film)
57     }
58
59     fun onWebButtonClicked(url: String) {
60         _event.value = Event.OpenWebUrl(url)
61     }
62
63     fun clearEvent() {
64         _event.value = null
65     }
66
67     sealed class Event {
68         data class NavigateToDetail(val film: Film) : Event()
69         data class OpenWebUrl(val url: String) : Event()
70     }
71 }

```

Tabel 5.1 6 Source Code FilmRepository.kt Jetpack Compose

FilmViewModelFactory.kt

```
1 package com.example.scrollablelist.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import com.example.scrollablelist.data.FilmRepository
6
7 class FilmViewModelFactory(
8     private val repository: FilmRepository
9 ) : ViewModelProvider.Factory {
10
11     override fun <T : ViewModel> create(modelClass: Class<T>):
12 T {
13
14     if (modelClass.isAssignableFrom(FilmViewModel::class.java)) {
15         @Suppress("UNCHECKED_CAST")
16         return FilmViewModel(repository) as T
17     }
18     throw IllegalArgumentException("Unknown ViewModel
19 class")
20 }
21 }
```

Tabel 5.1 7 Source Code FlmViewModelFactory.kt Jetpack Compose

HomeScreen.kt

```
1 package com.example.scrollablelist.screens
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.lazy.LazyColumn
8 import androidx.compose.foundation.lazy.items
9 import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.*
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.draw.clip
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.platform.LocalContext
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19 import androidx.lifecycle.viewmodel.compose.viewModel
20 import androidx.navigation.NavController
21 import coil.compose.rememberAsyncImagePainter
22 import com.example.scrollablelist.data.FilmRepository
23 import com.example.scrollablelist.network.RetrofitInstance
24 import com.example.scrollablelist.ui.FilmViewModel
25 import com.example.scrollablelist.ui.FilmViewModelFactory
26
27 @Composable
28 fun HomeScreen(navController: NavController) {
```

```

29     val context = LocalContext.current
30
31     val repository = remember {
32         FilmRepository(
33             api = RetrofitInstance.api,
34             apiKey = "9fad0dc9a0338ecf00596875e4cf5645"
35         )
36     }
37     val viewModel: FilmViewModel = viewModel(factory =
38 FilmViewModelFactory(repository))
39
40     val films by viewModel.filmList.collectAsState()
41     val event by viewModel.event.collectAsState()
42
43     val imageUrl = "https://image.tmdb.org/t/p/w500"
44
45     LaunchedEffect(event) {
46         when (event) {
47             is FilmViewModel.Event.NavigateToDetail -> {
48                 val film = (event as
49 FilmViewModel.Event.NavigateToDetail).film
50                 navController.navigate(
51
52 "detail/${Uri.encode(film.title)}/${Uri.encode(film.posterPath
53 ? : "")}/${Uri.encode(film.overview)}"
54                 )
55                 viewModel.clearEvent()
56             }
57             is FilmViewModel.Event.OpenWebUrl -> {
58                 val url = (event as
59 FilmViewModel.Event.OpenWebUrl).url
60                 val intent = Intent(Intent.ACTION_VIEW,
61 Uri.parse(url))
62                 navController.context.startActivity(intent)
63                 viewModel.clearEvent()
64             }
65             null -> Unit
66         }
67     }
68
69     LazyColumn(
70         contentPadding = PaddingValues(16.dp),
71         verticalArrangement = Arrangement.spacedBy(16.dp)
72     ) {
73         items(films) { film ->
74             Card(
75                 shape = RoundedCornerShape(16.dp),
76                 colors
77 CardDefaults.cardColors(containerColor = Color(0xFF2C2C2C)),
78                 modifier = Modifier.fillMaxWidth()
79             ) {
80                 Row(
81                     modifier = Modifier
82                         .padding(16.dp)

```



```

83         .fillMaxWidth()
84     ) {
85         Image(
86             painter
87             rememberAsyncImagePainter(imageBaseUrl + (film.posterPath ?:
88             "")),
89             contentDescription = film.title,
90             modifier = Modifier
91                 .size(100.dp)
92                 .clip(RoundedCornerShape(16.dp))
93         )
94
95         Spacer(modifier = Modifier.width(16.dp))
96
97         Column(
98             modifier = Modifier.weight(1f)
99         ) {
100             Text(
101                 text = film.title,
102                 style
103                 MaterialTheme.typography.titleMedium,
104                 color = Color.White,
105                 fontWeight = FontWeight.Bold,
106                 fontSize = 18.sp
107             )
108             Spacer(modifier
109             Modifier.height(4.dp))
110             Text(
111                 text = film.overview,
112                 style
113                 MaterialTheme.typography.bodyMedium,
114                 color = Color.Gray,
115                 maxLines = 3,
116                 overflow
117                 androidx.compose.ui.text.style.TextOverflow.Ellipsis
118             )
119
120             Spacer(modifier
121             Modifier.height(8.dp))
122
123             Row {
124                 Button(
125                     onClick
126                     viewModel.onItemClicked(film) },
127                     colors
128                     ButtonDefaults.buttonColors(
129                         containerColor
130                         Color(0xFFB3C7F9),
131                         contentColor = Color.Black
132                     ),
133                     shape
134                     RoundedCornerShape(50),
135                     modifier
136                     Modifier.height(40.dp)

```

```

137         ) {
138             Text(text = "Detail")
139         }
140     }
141 }
142 }
143 }
144 }
145 }
146 }

```

Tabel 5.1 8 Source Code HomeScreen.kt Jetpack Compose

MainScreen.kt

```

1 package com.example.scrollablelist.ui
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.clickable
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.lazy.LazyColumn
7 import androidx.compose.foundation.lazy.items
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.Composable
10 import androidx.compose.runtime.collectAsState
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.layout.ContentScale
13 import androidx.compose.ui.unit.dp
14 import coil.compose.rememberAsyncImagePainter
15 import com.example.scrollablelist.model.Film
16
17 @Composable
18 fun MainScreen(viewModel: FilmViewModel) {
19     // Ganti films ke filmList, dan pastikan collectAsState
20     // dengan tipe eksplisit
21     val films = viewModel.filmList.collectAsState(initial =
22     emptyList())
23
24     Scaffold(
25         topBar = {
26             SmallTopAppBar(title = { Text("Popular Movies") })
27         }
28     ) { padding ->
29         LazyColumn(
30             modifier = Modifier
31                 .padding(padding)
32                 .fillMaxSize()
33         ) {
34             items(films.value) { film ->
35                 FilmListItem(film)
36             }
37         }
38     }
39 }
40
41 @OptIn(ExperimentalMaterial3Api::class)

```

42	@Composable
43	fun SmallAppBar(title: @Composable () -> Unit) {
44	AppBar(title = title)
45	}
46	
47	@Composable
48	fun FilmListItem(film: Film) {
49	Row(
50	modifier = Modifier
51	.fillMaxWidth()
52	.clickable { /* Navigate to details if needed */ }
53	.padding(8.dp)
54	) {
55	Image(
56	painter
57	rememberAsyncImagePainter("https://image.tmdb.org/t/p/w500\${film.posterPath}"),
58	contentDescription = film.title,
59	modifier = Modifier.size(80.dp),
60	contentScale = ContentScale.Crop
61	)
62	Spacer(modifier = Modifier.width(16.dp))
63	Column(modifier = Modifier.fillMaxWidth()) {
64	Text(text = film.title, style =
65	MaterialTheme.typography.titleMedium)
66	Spacer(modifier = Modifier.height(4.dp))
67	Text(
68	text = film.overview,
69	maxLines = 3,
70	style = MaterialTheme.typography.bodyMedium
71	)
72	)
73	}
74	}
75	}

Table 5.1 9 Source Code mainScreen.kt Jetpack Compose

Mappers.kt

1	package com.example.scrollablelist.model
2	
3	fun NetworkFilm.toFilm(): Film {
4	return Film(
5	id = id,
6	title = title,
7	overview = overview,
8	posterPath = posterPath,
9	backdropPath = backdropPath,
10	releaseDate = releaseDate,
11	homepage = null
12	)
13	}

Table 5.1 10 Source Code MovieResponse.kt Jetpack Compose

MovieResponse.kt

```

1 package com.example.scrollablelist.model
2
3 fun NetworkFilm.toFilm(): Film {
4     return Film(
5         id = id,
6         title = title,
7         overview = overview,
8         posterPath = posterPath,
9         backdropPath = backdropPath,
10        releaseDate = releaseDate,
11        homepage = null
12    )
13 }

```

Tabel 5.1 11 Source Code MovieResponse.kt Jetpack Compose

MyApplication.kt

```

1 package com.example.scrollablelist
2
3 import android.app.Application
4 import
5 com.example.scrollablelist.data.FilmRepository
6 import com.example.scrollablelist.network.TmdbApi
7 import
8 com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
9 import kotlinx.serialization.json.Json
10 import okhttp3.MediaType.Companion.toMediaType
11 import okhttp3.OkHttpClient
12 import retrofit2.Retrofit
13 import timber.log.Timber
14
15
16 class MyApplication : Application() {
17
18     lateinit var filmRepository: FilmRepository
19     private set
20
21     lateinit var preferenceManager:
22 PreferenceManager
23     private set
24
25     override fun onCreate() {
26         super.onCreate()
27
28         Timber.plant(Timber.DebugTree())
29
30         preferenceManager = PreferenceManager(this)
31
32         val contentType =
33 "application/json".toMediaType()
34         val json = Json { ignoreUnknownKeys = true
35     }
36
37         val okHttpClient =
38 OkHttpClient.Builder().build()

```

```

39         val retrofit = Retrofit.Builder()
40
41         .baseUrl("https://api.themoviedb.org/3/")
42         .client(okHttpClient)
43
44         .addConverterFactory(json.asConverterFactory(contentType))
45         .build()
46
47         val api = retrofit.create(TmdbApi::class.java)
48
49         val apiKey = "9fad0dc9a0338ecf00596875e4cf5645"
50
51         filmRepository = FilmRepository(api, apiKey)
52     }
53 }

```

Tabel 5.1 12 Source Code MyApplication.kt Jetpack Compose

Navigation.kt

```

1 package com.example.scrollablelist.navigation
2
3 import androidx.compose.runtime.Composable
4 import androidx.navigation.NavHostController
5 import androidx.navigation.compose.NavHost
6 import androidx.navigation.compose.composable
7 import
8 androidx.navigation.compose.rememberNavController
9
10 import
11 com.example.scrollablelist.screens.DetailScreen
12 import
13 com.example.scrollablelist.screens.HomeScreen
14
15 @Composable
16 fun NavGraph(navController: NavHostController =
17 rememberNavController()) {
18     NavHost(
19         navController = navController,
20         startDestination = "home"
21     ) {
22         composable("home") {
23             HomeScreen(navController)
24         }
25
26         composable("detail/{title}/{imageUrl}/{fullDescription}") { backStackEntry ->
27             val title =
28             backStackEntry.arguments?.getString("title") ?:
29             ""
30
31

```

32	val	imageUrl	=
33	backStackEntry.arguments?.getString("imageUrl")		
34	?: ""		
35	val	fullDescription	=
36	backStackEntry.arguments?.getString("fullDescription")		
37	?: ""		
38			
39	DetailScreen(		
40	title = title,		
41	imageUrl = imageUrl,		
42	fullDescription	=	
43	fullDescription,		
44	onBack	=	{
45	navController.popBackStack() }		
46	)		
47	}		
48	}		
49	}		
50			

Table 5.1 13 Source Code Navigation.kt Jetpack Compose

NetworkFilm.kt

1	package com.example.scrollablelist.navigation
2	package com.example.scrollablelist.model
3	
4	import kotlinx.serialization.SerialName
5	import kotlinx.serialization.Serializable
6	import
7	kotlinx.serialization.InternalSerializationApi
8	import kotlin.annotation.AnnotationTarget.CLASS
9	
10	@OptIn(InternalSerializationApi::class)
11	@Serializable
12	data class NetworkFilm(
13	val id: Int,
14	val title: String,
15	val overview: String,
16	@SerialName("poster_path") val posterPath:
17	String?,
18	@SerialName("backdrop_path") val
19	backdropPath: String?,
20	@SerialName("release_date") val releaseDate:
21	String?
22	)

Table 5.1 14 Source Code NetworkFilm.kt Jetpack Compose

PreferenceManager.kt

1	package com.example.scrollablelist
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	import kotlinx.coroutines.flow.Flow
6	import kotlinx.coroutines.flow.MutableStateFlow

7	
8	class PreferenceManager(context: Context) {
9	
10	private val prefs: SharedPreferences =
11	
12	context.getSharedPreferences("app_prefs",
13	Context.MODE_PRIVATE)
14	
15	private val _isDarkMode =
16	MutableStateFlow(prefs.getBoolean(KEY_DARK_MODE
17	, false))
18	val isDarkMode: Flow<Boolean> = _isDarkMode
19	
20	fun setDarkMode(enabled: Boolean) {
21	prefs.edit().putBoolean(KEY_DARK_MODE,
22	enabled).apply()
23	_isDarkMode.value = enabled
24	}
25	
26	companion object {
27	private const val KEY_DARK_MODE =
28	"key_dark_mode"
29	}
30	}

Tabel 5.1 15 Source Code PreferenceManager.kt Jetpack Compose

Result.kt

1	package com.example.scrollablelist
2	
3	
4	sealed class Result<out T> {
5	object Loading : Result<Nothing>()
6	data class Success<T>(val data: T) :
7	Result<T>()
8	data class Error(val exception: Throwable) :
9	Result<Nothing>()
10	}

Tabel 5.1 16 Source Code Result.kt Jetpack Compose

RetrofitInstance.kt

1	package com.example.scrollablelist.network
2	
3	import
4	com.jakewharton.retrofit2.converter.kotlinx.ser
5	ialization.asConverterFactory
6	import kotlinx.serialization.json.Json
7	import okhttp3.MediaType.Companion.toMediaType
8	import okhttp3.OkHttpClient
9	import retrofit2.Retrofit
10	
11	object RetrofitInstance {
12	
13	private val json = Json {

```

14         ignoreUnknownKeys = true // biar bisa
15         ignore field yg gak dipakai
16     }
17
18     private val client = OkHttpClient.Builder()
19         .build()
20
21     private val retrofit by lazy {
22         Retrofit.Builder()
23
24         .baseUrl("https://api.themoviedb.org/3/") //
25         base URL TMDb API
26         .client(client)
27
28         .addConverterFactory(json.asConverterFactory("a
29 pplication/json".toMediaType()))
30         .build()
31     }
32
33     val api: TmdbApi by lazy {
34         retrofit.create(TmdbApi::class.java)
35     }
36 }

```

Tabel 5.1 17 Source Code RetrofitInstance.kt Jetpack Compose

SharedPreferencesViewModel.kt

```

1 package com.example.scrollablelist
2
3 import androidx.lifecycle.ViewModel
4 import kotlinx.coroutines.flow.StateFlow
5 import kotlinx.coroutines.flow.SharingStarted
6 import kotlinx.coroutines.flow.stateIn
7 import kotlinx.coroutines.flow.map
8 import kotlinx.coroutines.CoroutineScope
9 import kotlinx.coroutines.Dispatchers
10
11 class SharedPreferencesViewModel(private val
12 preferenceManager: PreferenceManager) :
13 ViewModel() {
14
15     val isDarkMode: StateFlow<Boolean> =
16 preferenceManager.isDarkMode
17     .stateIn(
18         CoroutineScope(Dispatchers.Main),
19         SharingStarted.Eagerly,
20         false
21     )
22
23     fun toggleDarkMode() {
24         val current = isDarkMode.value
25         preferenceManager.setDarkMode(!current)
26     }
27 }

```

Tabel 5.1 18 Source Code SharedPreferencesViewModel.kt Jetpack Compose

SharedPreferenceViewModelFactory.kt

```
1 package com.example.scrollablelist
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5
6 class SharedPreferenceViewModelFactory(
7     private val preferenceManager:
8     PreferenceManager
9 ) : ViewModelProvider.Factory {
10
11     override fun <T : ViewModel>
12     create(modelClass: Class<T>): T {
13         if
14         (modelClass.isAssignableFrom(SharedPreferenceVi
15         ewModel::class.java)) {
16             @Suppress("UNCHECKED_CAST")
17             return
18             SharedPreferenceViewModel(preferenceManager) as
19             T
20         }
21         throw IllegalArgumentException("Unknown
22         ViewModel class")
23     }
24 }
```

Table 5.1 19 Source Code SharedPreferenceViewModelFactory.kt Jetpack Compose

TMDBApi.kt

```
1 package com.example.scrollablelist.network
2
3 import
4 com.example.scrollablelist.model.MovieResponse
5 import retrofit2.http.GET
6 import retrofit2.http.Query
7
8 interface TmdbApi {
9     @GET("movie/popular")
10     suspend fun getPopularMovies(
11         @Query("api_key") apiKey: String
12     ): MovieResponse
13 }
```

Table 5.1 20 Source Code TMDBApi.kt Jetpack Compose

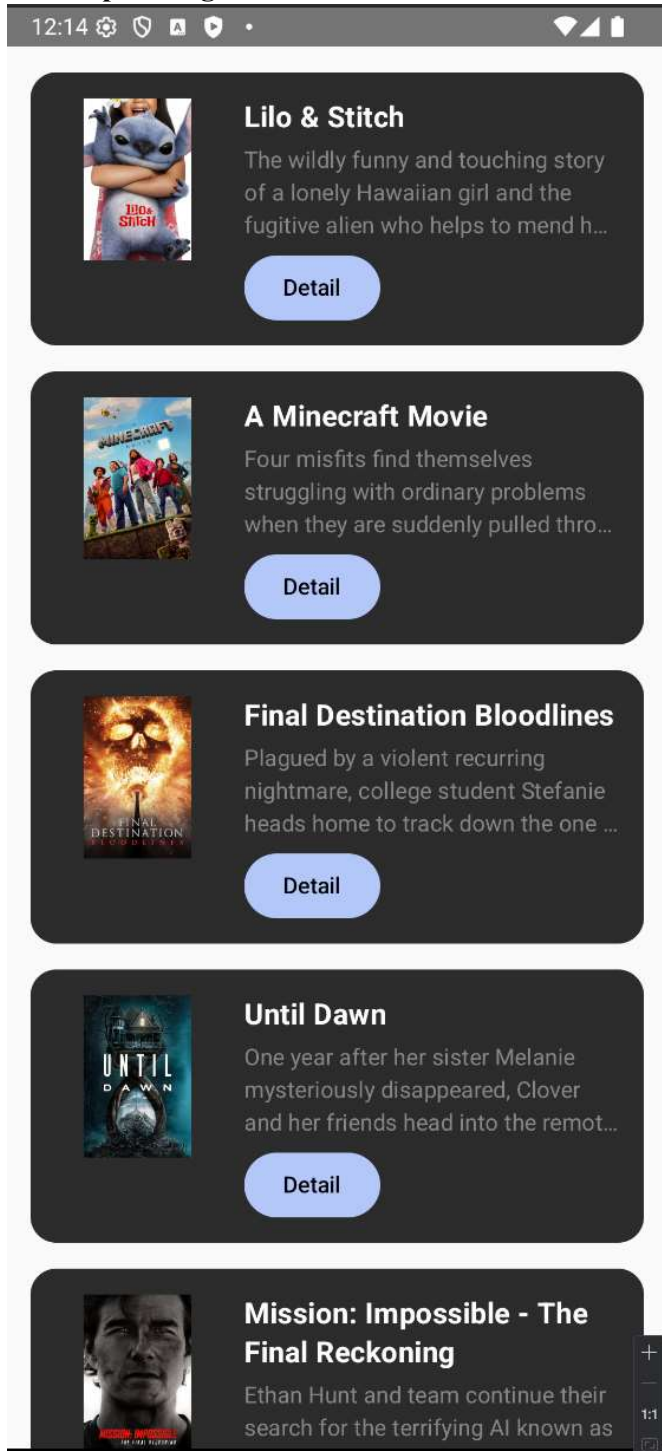
SettingsScreen.kt

```
1 package com.example.scrollablelist.screens
2
3 import androidx.compose.material3.Switch
4 import androidx.compose.material3.Text
5 import androidx.compose.foundation.layout.Row
6 import
7 androidx.compose.foundation.layout.padding
8 import androidx.compose.runtime.Composable
```

9	import androidx.compose.runtime.collectAsState
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.unit.dp
12	import
13	com.example.scrollablelist.SharedPreferenceView
14	Model
15	
16	@Composable
17	fun SettingsScreen(viewModel:
18	SharedPreferenceViewModel) {
19	val isDarkMode =
20	viewModel.isDarkMode.collectAsState()
21	
22	Row(modifier = Modifier.padding(16.dp)) {
23	Text(text = "Dark Mode")
24	Switch(
25	checked = isDarkMode.value,
26	onCheckedChange = {
27	viewModel.toggleDarkMode() },
28	modifier = Modifier.padding(start =
29	8.dp)
30	)
31	}
32	}

Tabel 5.1 21 Source Code SettingScreen.kt Jetpack Compose

B. Output Program



Gambar 5.1 1 Screenshot Hasil Jawaban Soal 1 Modul 5

C. Pembahasan

MainActivity.kt Jetpack Compose:

- `MainActivity` adalah titik awal aplikasi.
- `setContent` digunakan untuk menampilkan UI berbasis **Jetpack Compose**.
- `ScrollableListTheme { NavGraph() }` memastikan bahwa seluruh UI mengikuti tema tertentu dan menggunakan sistem navigasi terpisah (`NavGraph`) untuk berpindah antar halaman (screen).
- `apiKey` disimpan secara lokal, sebaiknya diletakkan di `BuildConfig` atau `local.properties` untuk keamanan.

ApiService.kt

- Menggunakan **Retrofit** untuk mengambil data dari endpoint `movie/popular` di TMDB.
- Fungsi `suspend` artinya dapat dipanggil dalam coroutine (asynchronous).
- Menggunakan query `api_key` untuk otorisasi.
- Return type adalah `MovieResponse`, yaitu response model JSON dari TMDB.

Film.kt Jetpack Compose :

- `Film` adalah model yang digunakan dalam **lapisan domain** (digunakan oleh UI dan repository).
- Disusun agar bersih dari detail jaringan (network response), memudahkan testing dan pemeliharaan.
- Dipisahkan dari model response agar modular dan tidak terikat pada struktur response TMDB.

FilmRepository.kt Jetpack Compose :

- Menghubungkan data dari `api` ke dalam model `Film`.
- `api.getPopularMovies(apiKey)` adalah panggilan ke Retrofit.
- Kemudian, hasilnya dikonversi dari response model ke domain model `Film`.

DetailScreen.kt

- UI detail film, menampilkan **judul**, **gambar**, dan **deskripsi** film.
- Gunakan **Coil** (`rememberAsyncImagePainter`) untuk memuat gambar dari URL TMDB.
- Menggunakan `Scaffold` dan `SmallTopAppBar` untuk UI yang konsisten dan material-based.
- `onBack` digunakan agar tombol back bisa kembali ke screen sebelumnya dengan `BackHandler`.

FilmViewModel.kt

Fungsi Utama:

- Mengelola **state film** menggunakan `StateFlow` agar Compose bisa observe data secara reaktif.
- Menangani navigasi dan aksi user dengan event handler berbasis sealed class (`Event`).

Penjelasan Teknis:

- `_filmList` menyimpan daftar film dan diamati dari UI menggunakan `collectAsState()`.
- `_event` digunakan sebagai kanal komunikasi satu arah (event-driven) ke UI, seperti `NavigateToDetail` atau `OpenWebUrl`.
- `viewModelScope.launch {}` memungkinkan coroutine berjalan lifecycle-aware (tidak memory leak).
- `loadFilmsFromApi()` memanggil repository dan menangani error log dengan `Timber`.
-

FilmViewModelFactory.kt

Fungsi Utama:

- Membuat instance dari `FilmViewModel` dengan repository sebagai dependensi eksternal (dependency injection manual).

Penjelasan Teknis:

- Diperlukan karena `FilmViewModel` tidak memiliki constructor kosong.
- Digunakan di `HomeScreen` untuk membuat `ViewModel` secara benar.

HomeScreen.kt

Fungsi Utama:

- Menampilkan daftar film populer.
- Meng-handle klik tombol Detail dan Web dengan Event dari `ViewModel`.
- Menginisialisasi repository dan `ViewModel` (biasanya bisa dikelola oleh DI seperti Hilt).

Penjelasan Teknis:

- `LaunchedEffect(event)` digunakan untuk menjalankan navigasi satu kali saat event terjadi.
- Navigasi dilakukan dengan `navController.navigate(...)`.
- Gambar dimuat menggunakan `Coil (rememberAsyncImagePainter)`.
- List film ditampilkan menggunakan `LazyColumn`.

MainScreen.kt

Fungsi Utama:

- Menampilkan daftar film seperti `HomeScreen`, namun tidak memuat tombol dan event detail.
- Tampaknya merupakan screen awal atau versi sederhana dari tampilan utama.

Penjelasan Teknis:

- Menggunakan `LazyColumn` untuk menampilkan list.
- Tidak ada aksi klik di `FilmListItem` (komentar placeholder).
- Gunakan `rememberAsyncImagePainter` untuk gambar (via `Coil`).

Mappers.kt

Fungsi Utama:

- Mengonversi dari `NetworkFilm` (model response API) ke `Film` (domain model).

Penjelasan Teknis:

- Memisahkan model dari API dengan model domain merupakan praktik Clean Architecture agar layer UI dan repository tidak terikat dengan detail dari API.
- Digunakan di `FilmRepository`.

MovieResponse.kt

Penjelasan:

- Fungsi ekstensi untuk mengonversi `NetworkFilm` (response dari API) menjadi `Film` (model domain).
- Ini penting karena kamu memisahkan data yang didapat dari internet (`NetworkFilm`) dengan data yang digunakan di aplikasi (`Film`), sebuah prinsip Clean Architecture.

MyApplication.kt

Isinya:

- Inisialisasi global seperti `FilmRepository` dan `PreferenceManager`.
- Konfigurasi Retrofit menggunakan Kotlin Serialization.
- Setup Timber untuk logging.

Penjelasan:

- File ini menjalankan `Application`, yang hanya dibuat sekali saat app dijalankan.
- `filmRepository` dan `preferenceManager` disiapkan di sini agar bisa digunakan di mana saja, misalnya disuntikkan ke `ViewModel`.
- `Json { ignoreUnknownKeys = true }` membuat deserializer lebih fleksibel terhadap perubahan API.

NetworkFilm.kt

Penjelasan:

- `@Serializable`: Mengaktifkan serialisasi/deserialisasi dengan KotlinX Serialization.
- `@SerializedName(...)`: Menghubungkan nama di JSON dengan properti Kotlin.
- Ini adalah representasi data mentah dari TMDb API.

Navigation.kt

Isinya:

- Menentukan flow navigasi antara dua screen: `HomeScreen` dan `DetailScreen`.
- Gunakan `NavHost` dari Jetpack Compose Navigation.

Penjelasan:

- `NavHost` mendefinisikan struktur screen.
- Data dari `HomeScreen` ke `DetailScreen` dikirim lewat path parameter (`detail/{title}/{imageUrl}/{fullDescription}`).
- `Uri.encode(...)` di `HomeScreen` penting agar teks tidak rusak saat dikirim melalui URL.

PreferenceManager.kt

Isinya:

- Membungkus `SharedPreferences` agar bisa digunakan secara reaktif menggunakan `StateFlow`.
- Menyimpan dan membaca mode gelap/terang (dark mode).

Penjelasan:

- `MutableStateFlow` memungkinkan UI bereaksi jika user mengaktifkan atau mematikan dark mode.
- Menyediakan API `setDarkMode(enabled)` dan `isDarkMode: Flow<Boolean>`.

Result.kt

Fungsi:

- Membungkus status dari operasi async (misal: memanggil API).

Penjelasan:

`sealed class Result<out T>` memiliki tiga kemungkinan:

- `Loading`: Proses sedang berlangsung.
- `Success<T>(val data: T)`: Berhasil dan menyimpan hasil.
- `Error(val exception: Throwable)`: Gagal dan menyimpan error-nya.

RetrofitInstance.kt

Fungsi:

- Singleton untuk membuat instance Retrofit dan `TmdbApi`.

Penjelasan:

- `Json { ignoreUnknownKeys = true }`: Menghindari crash jika JSON API punya field yang tidak didefinisikan di model.
- `by lazy`: Menunda inisialisasi sampai digunakan.

SharedPreferenceViewModel.kt

Fungsi:

- Mengelola dan mengekspos status dark mode dari `PreferenceManager`.

Penjelasan:

- `stateIn(...)`: Mengubah Flow jadi `StateFlow`, sehingga bisa digunakan di Compose dan Live UI update.
- `toggleDarkMode()`: Membalik status dark mode dan menyimpannya.

SharedPreferenceViewModelFactory.kt

Fungsi:

- Membuat instance `SharedPreferenceViewModel` dengan constructor yang membutuhkan parameter (`PreferenceManager`).

Penjelasan:

- Karena `ViewModelProvider` default hanya bisa buat `ViewModel` tanpa argumen, kamu butuh factory ini.

TmdbApi.kt

Fungsi:

- Interface untuk endpoint Retrofit ke API TMDb.

Penjelasan:

- Endpoint: `GET https://api.themoviedb.org/3/movie/popular?api_key=...`
- Hasil dikembalikan sebagai `MovieResponse` (yang berisi list `NetworkFilm`).

SettingsScreen.kt

Fungsi:

- Tampilan UI untuk mengubah mode gelap/terang dengan Switch.

Penjelasan:

- Menggunakan `collectAsState()` agar UI auto update saat state `isDarkMode` berubah.

- Ketika Switch diklik, `viewModel.toggleDarkMode()` dipanggil.

E. Tautan GIT

<https://github.com/gr1ff0m/Pemrograman-Mobile-Praktikum>